

REAL-TIME HUMAN POSE CORRECTION SYSTEM USING TEMPORAL ANALYSIS AND ACTIONABLE GUIDANCE

A PROJECT REPORT

Submitted By

NANDHA S	-	510822243010
RAGUL G	-	510822243021
UDHAYA P	-	510822243027

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE & DATA SCIENCE



GANADIPATHY TULSI'S JAIN ENGINEERING COLLEGE

KANIYAMBADI, VELLORE-632102.

ANNA UNIVERSITY :: CHENNAI 600 025

MAY 2026

BONAFIDE CERTIFICATE

Certified that this project report “**REAL-TIME HUMAN POSE CORRECTION SYSTEM USING TEMPORAL ANALYSIS AND ACTIONABLE GUIDANCE**” is the bonafide work of “**NANDHA S [510822243010], RAGUL G [510822243021]** and **UDHAYA P [510822243027]**” who carried out the project work under my supervision.

SIGNATURE

E LAVANYA

SUPERVISOR

Assistant Professor

Department of Artificial Intelligence and Data Science

Ganadipathy Tulsi's Jain Engineering College

Kaniyambadi, Vellore-632 102.

SIGNATURE

S PRIYA

HEAD OF THE DEPARTMENT

Assistant Professor

Department of Artificial Intelligence and Data Science

Ganadipathy Tulsi's Jain Engineering College

Kaniyambadi, Vellore-632 102.

Submitted for the Project Viva-Voce Examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We extend our sincere gratitude to the divine for granting us the resilience and fortitude to successfully complete our Project work.

We deeply appreciate the unwavering support provided by our esteemed College Managing Trustee **Mr. N. Sugul Chand Jain**, Secretary **Mr. T. Amar Chand Jain**, our esteemed Principal **Dr. M. Barathi**, and Respected Vice Principal **Mr. D. Durai Kumar**.

We own our profound gratitude to our Head of the Department **Mrs. S. Priya**, Assistant Professor, Department of Artificial Intelligence and Data Science for her valuable guidance, suggestions and constant encouragement that paved way for the successful completion of the project work.

We sincerely thank **Mrs. E. Lavanya**, Assistant Professor, for her valuable guidance, support, and encouragement throughout our project.

We also acknowledge and thank all faculty members of the Department Artificial Intelligence and Data Science for their kind assistance and for facilitating the necessary resources for the project.

Finally, we express our sincere gratitude to our families and friends for their unwavering support and encouragement throughout this endeavor.

ABSTRACT

Human pose correction is crucial in exercise training, yoga, sports coaching, and rehabilitation, as improper posture can reduce effectiveness and cause injury. Existing pose correction systems primarily rely on recorded images or videos, providing feedback only after the exercise, which often results in an overwhelming number of frame-by-frame instructions that are difficult for users to interpret and apply in real time. To address these limitations, this project proposes a real-time human pose correction system that uses a live camera feed and temporal analysis to continuously monitor body posture during exercise. By tracking skeletal landmarks across consecutive frames, the system identifies persistent posture deviations while filtering out temporary motion noise. Rather than overloading users with excessive instructions, it prioritizes critical errors and delivers simplified, actionable guidance through visual or audio cues. This approach allows users to correct their posture instantly, enhancing safety, accuracy, and learning effectiveness. The system is designed for practical applications in fitness, yoga, physiotherapy, and sports coaching, providing a user-friendly and efficient solution that overcomes the shortcomings of post-recorded pose correction methods.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	iv
	LIST OF FIGURES	viii
	LIST OF ABBREVIATIONS	ix
1	INTRODUCTION	1
	1.1. OBJECTIVE	1
	1.2. OVERVIEW	1
2	LITERATURE SURVEY	3
	2.1. EXISTING SYSTEM	11
	2.2. PROBLEM IDENTIFICATION	12
	2.2. PROBLEM ANALYSIS	13
	2.3. PROPOSED SYSTEM	14
3	SYSTEM SPECIFICATION	15
	3.1. HARDWARE REQUIREMENTS	15
	3.2. SOFTWARE REQUIREMENTS	15
	3.3. SOFTWARE DESCRIPTION	15
	3.3.1. Python	15
	3.3.2 Python Features	16
	3.4. DEEP LEARNING	20
	3.4.1. LSTM Network	20

4	SYSTEM DESIGN	23
	4.1 SYSTEM ARCHITECTURE	23
	4.2 DATA FLOW DIAGRAM	25
5	PROJECT DESCRIPTION	28
	5.1 MODULES	28
	5.2 MODULE DESCRIPTION	28
	5.2.1 Data Acquisition Module	28
	5.2.2 pose Estimation Module	29
	5.2.3 Feature Extraction Module	29
	5.2.4 Temporal Analysis Module	29
	5.2.5 Calibration Module	29
	5.2.6 Final Decision Module	30
	5.2.7 Feed Back Module	30
6	SYSTEM IMPLEMENTATION	31
	6.1 DATA ACQUISITION	31
	6.2 POSE ESTIMATION	32
	6.3 FEATURE EXTRACTION	33
	6.4 TEMPORAL FILTER	34
	6.5 CALIBRATION	35
	6.6 FINAL DECISION	37
	6.7 FEED BACK GENERATION	38

7	SYSTEM TESTING	40
	7.1. TESTING	40
	7.2. LEVELS OF TESTING	40
	7.2.1. Unit Testing	41
	7.2.2. Integration Testing	41
	7.2.3. Functional Testing	42
	7.2.4. Performance Testing	42
8	CONCLUSION	43
	APPENDICES	44
	APPENDIX 1: SOURCE CODE	44
	APPENDIX 2: SCREEN SHOTS	62
	BIBLIOGRAPHY	70

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
4.1	SYSTEM ARCHITECTURE	23
4.2	DATA FLOW DIAGRAM	25

LIST OF ABBREVIATIONS

CNN	Convolutional Neural Network
LSTM	Long Short-Term Memory
Bi-CGRU	Bidirectional Convolutional Gated Recurrent Unit
Bi-GRU	Bidirectional Gated Recurrent Unit
IMU	Inertial Measurement Unit
RFL	Random Forest Long Short-Term Memory
GRE	Geometric Representation Extraction

CHAPTER 1

INTRODUCTION

1.1 OBJECTIVE

The objective of the proposed system is to create an intelligent real-time human pose monitoring framework that detects and tracks body movements, identifies posture deviations and joint misalignments, delivers instant and actionable correction guidance, prioritizes critical errors to improve user comprehension, and ultimately enhances exercise safety, posture accuracy, and overall performance.

1.2 OVERVIEW

Maintaining proper body posture during physical activities such as yoga, fitness training, sports, and rehabilitation is critical for maximizing effectiveness and preventing injuries. Automated human pose analysis has become a key focus in AI-powered healthcare and fitness applications. Pose estimation algorithms, such as Blaze Pose from Media Pipe, allow machines to detect and track skeletal landmarks, including joints and limbs from live video feeds or images. These keypoints form the basis for analyzing posture correctness and generating corrective feedback in real time.

Existing pose correction systems are often offline and frame-based, analyzing recorded videos or images after the exercise session. They compare user poses with predefined ideal examples, generating numerous corrective instructions that are difficult to interpret and apply during dynamic movements. This frame-by-frame approach lacks temporal understanding, making the system prone to false corrections due to transient movements or camera noise. Consequently, users cannot correct their posture immediately, increasing the risk of repeated mistakes and potential injuries.

To address these limitations, the proposed system introduces a real-time human pose correction framework that leverages temporal analysis and machine learning. Skeletal landmarks are tracked across multiple consecutive frames, and joint angles are calculated using vector geometry to quantify posture deviations. An MLP-based classifier then determines whether a posture is correct or incorrect, and a rule-based feedback engine generates simplified, prioritized textual instructions for the user. This pipeline ensures consistent error detection, real-time actionable guidance, and improved usability, making the system particularly suitable for rehabilitation, physiotherapy, yoga practice, and fitness training.

CHAPTER 2

LITERATURE SURVEY

[1] A. Dittakavi et al. "PoseGuru: Landmarks for Explainable Pose Correction using Exemplar-Guided Algorithmic Recourse (CVPRW-2025)".

This project proposes a human pose correction system, PoseGuru, designed for applications in fitness, rehabilitation, and sports. PoseGuru provides explainable, personalized, and fine-grained corrections for human body poses by leveraging algorithmic recourse and exemplar-guided optimization. The system detects misaligned joints in an individual's pose and iteratively generates corrected landmarks that adhere to user-specific anatomical constraints. PoseGuru uses 2D images and does not require expensive 3D setups, making it scalable for mobile and edge devices. The system introduces new datasets, YogaHPC and Pilates32+P, for robust evaluation of pose correction methods.

Pros:

- Delivers user-specific correction respecting anatomical constraints.
- Offers explainable feedback for understanding adjustments.
- Generates corrected landmarks for progress tracking.

Cons:

- Limited to 2D without depth perception.
- Susceptible to noisy landmark extraction errors.
- Requires manual selection of optimal exemplars.
- Misses subtle misalignments in complex poses.

[2] Lei Yang, Yingxiang Li, Degui Zeng, Dong Wang. "Human Exercise Posture Analysis based on Pose Estimation (IEEE-2021)".

This project proposes a human exercise posture analysis software designed to detect and analyze exercise postures such as squats and push-ups using video input. The system uses OpenPose as the key pose estimation network and enhances it to improve detection accuracy in special scenarios (e.g., occlusion, non-upright people). The software extracts key point coordinates, regularizes them according to body proportions, and compares user performance against defined standard forms of exercise to provide posture correction guidance.

Pros:

- Real-time exercise posture analysis using video without wearable devices
- Accurate key point detection enhanced for occlusion and abnormal positions
- Provides text-based corrective feedback for users on exercises like squats and push-ups
- Works with videos from any standard recording device

Cons:

- Only supports 2 types of exercises (squats and push-ups) with predefined rules
- Feedback is text-only, lacking voice or visual guidance
- Requires users to record videos from specific perspectives for accurate detection
- Limited to 2D analysis, cannot capture depth information like 3D systems
- Model performance may degrade in crowded or complex backgrounds

[3] Ali Bidaran, Saeed Sharifian. "Designing an AI-assisted toolbox for fitness activity recognition based on deep CNN (IEEE-2021)".

This project proposes an AI-assisted toolbox for detecting fitness activities from video streams in virtual classrooms or online sessions. The system uses a two-stream convolutional neural network (CNN) with a MobileNetV2 backbone to process RGB frames (spatial features) and Magnitude Flow images (motion features). An Observer network based on Extreme Learning Machine (ELM) architecture fuses the predictions from both streams to improve accuracy and reduce uncertainties. The toolbox is designed to assist fitness instructors in tracking and analyzing multiple participants' performance remotely.

Pros:

- Combines spatial (RGB) and temporal features for higher detection accuracy
- Lightweight MobileNetV2 backbone enables deployment on edge device
- Provides quantitative accuracy metrics for each detected exercise

Cons:

- Tested on only 7 fitness activities; may not generalize to other exercise
- Confusing exercises with similar motions (e.g., Lunges vs Squats)
- preprocessing to compute optical flow that may add latency in real-time use
- Does not perform precise joint/keypoint detection like Mediapipe.
- Multi-person tracking is limited; accuracy may drop in crowded scenarios.

[4] Wan-Jung Chang, Jian-Ping Su, Chia-Hao Chen, et al. "iFitness: A Deep Learning-Based Physical Fitness Motion Detection System for Elderly People (IEEE-2021)".

This project proposes a deep learning-based physical fitness motion detection system, iFitness, for elderly people in community and home environments. The system consists of a camera, an AI edge module, and a fitness management platform. The camera captures video frames of the elderly performing fitness exercises, which are sent to the AI edge module. MediaPipe is used for pose estimation, extracting body landmarks and joint coordinates. A rule-based algorithm processes these landmarks to recognize exercises (Chair Stand, Arm Curl, Step Exercise) and count repetitions in real time. The fitness management platform collects the results, allowing doctors or rehabilitations to evaluate exercise effectiveness.

Pros:

- Allows elderly to exercise without a wearable device or constant supervision
- Edge-based processing ensures real-time performance and offline operation
- Privacy-friendly, no video leaves the edge device
- Lightweight rule-based logic suitable for low-power AI edge devices

Cons:

- Limited to certain exercises (S1, S2, S4); S3, S5, S6 not yet implemented
- Sensitive to user variation (height, limb length, mobility range)
- Performance affected by camera placement, lighting, and occlusion
- Recognition is rule-based; cannot adapt to new or complex exercises

[5] Prashant Rao, C. S. Asha, P. Raghavendra Rao. "Real-Time Posture Correction of Squat Exercise: A Deep Learning Approach for Performance Analysis and Error Correction (IEEE-2024)".

This study presents a real-time computer vision system that evaluates squat performance using MediaPipe for skeletal landmark extraction. It employs a Bidirectional Convolutional Gated Recurrent Unit (Bi-CGRU) classification model combined with a pose correction regression model to categorize squat depth and provide biomechanically guided posture corrections.

Pros:

- Achieves a high classification accuracy of 96.1%.
- Provides precise, biomechanics-based posture corrections.
- Evaluates exact squat depth rather than just counting reps.

Cons:

- Requires a multi-camera (stereo) setup for best results.
- Difficult to deploy on standard single-camera mobile devices.
- Bi-CGRU processing creates high computational overhead.

[6] Aisha Naseer, Ali Raza, Hadeeqa Afzal, et al. "Human Pose Estimation in Physiotherapy Fitness Exercise Correction Using Novel Transfer Learning Approach (PeerJ-2024)".

This open-access research introduces a hybrid Random Forest Long Short-Term Memory (RFL) method for physical therapy exercise correction. It extracts temporal features using an LSTM network and probabilistic features using a Random Forest model, processing massive amounts of time-series data to accurately classify and correct rehabilitation movements.

Pros:

- Achieves an outstanding 99% accuracy rate for clinical use.
- Highly robust at tracking temporal movement patterns.
- Efficiently processes complex time-series data.

Cons:

- Relies heavily on wearable IMU sensors.
- Lacks the convenience of a vision-only (camera) setup.
- Extra hardware makes it expensive for home users.

[7] Ulises Gallardo, Fernando Caro, Eluney Hernández, et al. "GYMetricPose: A Light-Weight Angle-Based Graph Adaptation for Action Quality Assessment (IEEE-2024)".

This paper proposes a cross-domain method to exploit geometric angle information in human pose skeletons for complex gym environments. It integrates a Geometric Representation Extraction (GRE) module with transformer-based pose estimation to detect fine-grained posture problems, specifically targeting "Knee Inward Errors" and "Shallow Squat Errors".

Pros:

- Highly lightweight with a small parameter footprint.
- Performs well despite cluttered gym backgrounds.
- Specifically targets fine-grained geometric errors.

Cons:

- Transformer architecture still introduces inference latency.
- Struggles to run smoothly on low-end edge devices.
- Geared toward general fitness rather than personalized rehab.

[8] Mukundan Chariar, Shreyas Rao, Aryan Irani, et al. "AI Trainer: Autoencoder Based Approach for Squat Analysis and Correction (IEEE-2023)".

This IEEE paper introduces a squat analysis system that uses MediaPipe for 2D pose estimation paired with an autoencoder to identify abnormal postures. A Bidirectional Gated Recurrent Unit (Bi-GRU) equipped with an attention mechanism models the motion dynamics, allowing the system to distinguish between correct and incorrect squats and generate real-time feedback.

Pros:

- Adapts easily to varying human body types and limb lengths.
- Successfully avoids using rigid, hard-coded angle thresholds.
- Generates highly effective real-time corrective feedback.

Cons:

- Autoencoders require massive datasets of "perfect form" to train.
- Difficult to adapt the model to entirely new exercises.
- Running dual networks (Autoencoder + Bi-GRU) drains CPU resources.

2.1 EXISTING SYSTEM

Existing human pose correction systems are designed to analyze human posture during exercises using recorded videos or images. These systems employ 2D pose estimation techniques such as OpenPose or MediaPipe to detect skeletal landmarks including joints and limbs. The posture is then compared with predefined standard poses to identify misalignments or deviations. Feedback is usually provided after the exercise is completed, often on a frame-by-frame basis, either as text instructions or visual cues highlighting errors. While these systems can accurately detect pose deviations, they operate offline, lack temporal understanding, and can overwhelm users with excessive instructions, making real-time correction impossible. Furthermore, many systems are limited to specific exercises, cannot adapt automatically to individual variations, and lack depth perception for subtle posture errors.

Pros:

- Can accurately detect joint misalignments and posture deviations.
- Works with standard cameras or recorded videos without wearables.
- Provides frame-by-frame analysis for detailed feedback.
- Useful for post-exercise performance tracking and evaluation.

Cons:

- Feedback is delayed; users cannot correct posture instantly.
- Frame-by-frame analysis may produce false corrections.
- Excessive instructions can overwhelm and confuse users.
- Limited to specific exercises; cannot generalize to all movements.
- Does not adapt automatically to user-specific variations.

2.2 PROBLEM IDENTIFICATION

Maintaining correct body posture during exercises, yoga, sports, or rehabilitation is critical for safety, effectiveness, and performance. Existing human pose correction systems mostly operate on recorded videos or images, providing feedback after the activity is completed. This offline, frame-by-frame feedback often generates many instructions that are difficult for users to interpret and apply in real time. Moreover, these systems lack temporal analysis, treating each frame independently, which can result in false or unstable corrections due to temporary movements or camera noise. Consequently, users cannot correct their posture instantly, increasing the risk of repeating incorrect movements and sustaining injuries.

There is a clear need for a real-time human pose correction system that continuously monitors body posture, identifies consistent deviations, filters out temporary motion noise, and delivers simplified, actionable feedback, enabling users to correct their posture immediately and safely.

2.3 PROBLEM ANALYSIS

Current pose correction systems primarily operate offline, analyzing recorded media only after an activity concludes, which prevents instant posture adjustments and actively increases the user's risk of injury. Furthermore, these systems rely on independent, frame-by-frame analysis lacking crucial temporal context. Without sequence tracking, minor physical shifts or camera noise frequently trigger false corrections, while this granular comparison generates an excessive volume of instructions that cognitively overload the user. Additionally, most implementations are rigidly limited to a narrow set of predefined exercises and fail to adapt to user-specific physical variations, such as different body types or mobility ranges. Finally, the reliance on standard 2D estimation severely restricts depth perception, causing systems to completely miss subtle postural misalignments and highlighting the urgent need for a continuous, noise-filtering, real-time system that delivers simplified, actionable feedback.

2.4 PROPOSED SYSTEM

The proposed system is a real-time human pose correction framework designed to facilitate continuous kinematic monitoring and corrective guidance through a synchronous video-stream pipeline integrated with Deep Temporal Analysis. Unlike conventional high-latency offline systems, this framework leverages a decentralized processing approach to maintain high-frequency tracking of 33 distinct skeletal landmarks across a sliding window of consecutive frames, effectively mapping anatomical joints in three-dimensional space. By employing an LSTM-based network trained on the UI-PRMD dataset, the system executes a Diagnostic Logic Protocol that synthesizes sequential biomechanical data against calibrated dynamic thresholds to identify persistent postural deviations while filtering out stochastic sensor noise. The resulting output is delivered via a Multi modal Feedback Interface, providing high-priority visual overlays and low-latency audio cues designed to prevent cognitive saturation and enable instantaneous proprioceptive correction for improved safety and performance in rehabilitation.

Pros:

- Real-time monitoring allows instant posture correction.
- Temporal tracking reduces false corrections caused by temporary motion or noise.
- Provides simplified, prioritized guidance, avoiding instruction overload.
- Improves safety, exercise accuracy, and learning effectiveness.
- Scalable and cost-effective, suitable for mobile or edge deployment.

CHAPTER 3

SYSTEM SPECIFICATION

3.1 HARDWARE REQUIREMENTS

- Processor : Intel Core i3(8th Generation)/ AMD Ryzen 3
- RAM : 8 GB
- Storage : 256 GB HDD / SSD
- Camera : Built-in Laptop Webcam (minimum 720p)
- GPU : Not required (CPU-based execution)
- Display : 14-inch or higher

3.2 SOFTWARE REQUIREMENTS

- Operating System : Windows 10 / Windows 11
- Programming Language : Python 3.8 – 3.11
- Development Environment : VS Code / PyCharm

3.3 SOFTWARE DESCRIPTION

3.3.1 Python

Python is a widely used general-purpose, high-level programming language. It was initially designed by Guido van Rossum in 1991 and developed by Python Software Foundation. It was mainly developed for emphasis on code readability, and its syntax allows programmers to express concepts in fewer lines of code. Python is a programming language that lets you work quickly and integrate systems more efficiently. It is used for:

- Software development
- Web development (server-side)
- Mathematics
- System scripting

3.3.2 Python Features

- Easy to learn, python has few keywords, a simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- Easy to read, python code is more clearly defined and visible to the eyes.
- Easy to maintain, python's source code is easy-to-maintain.
- Abroad standard library, python's bulk of the library is very portable and cross-platform compatible on UNIX, Wind 16 .
- Portable, python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- Interactive Mode, python has support for an interactive mode that allows interactive testing and debugging of snippets of code.
- Extendable, you can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient. Databases, python provides interfaces to all major commercial databases.
- GUI Programming, python supports GUI applications that can be created and ported to many system calls, libraries, and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.
- Scalable, python provides a better structure and support for large programs than shell scripting

Python Libraries

- Tensor Flow
- Media Pipe
- OpenCV
- Numpy
- OS
- CSV
- Streamlit

- Time
- Keras
- Subprocess
- Sys

Tensor Flow

Tensor Flow is a very popular open-source library for high-performance numerical computation developed by the Google Brain team in Google. As the name suggests, Tensor Flow is a framework that involves defining and running computations involving tensors. It can train and run deep neural networks that can be used to develop several AI applications. Tensor Flow is widely used in the field of deep learning research and application.

Media Pipe

MediaPipe is an open-source framework developed by Google that is used to build real-time machine learning pipelines for processing video, audio, and sensor data. It provides ready-to-use, pre-trained models for tasks such as face detection, hand tracking, pose estimation, and object detection. MediaPipe is optimized to run efficiently on mobile devices, web applications, and desktop systems, making it widely used in computer vision projects like gesture recognition, augmented reality filters, fitness tracking, and AI-based pose detection systems.

OpenCV

OpenCV (Open Source Computer Vision Library) is a widely used open-source computer vision and machine learning library. It provides a comprehensive suite of tools and functions for real-time computer vision tasks, including image and video processing, feature detection, object recognition, and image manipulation. Written in C++ with Python bindings, OpenCV is highly optimized for performance, making it suitable for applications in robotics, surveillance, augmented reality, and artificial intelligence. Its extensive library and versatility make it a popular choice for developers working with visual data.

NumPy

NumPy is a very popular Python library for large multi-dimensional array and matrix processing, with the help of a large collection of high-level mathematical functions. It is very useful for fundamental scientific computations in Machine Learning. It is particularly useful for linear algebra, Fourier transform, and random number capabilities. High-end libraries like TensorFlow use NumPy internally for the manipulation of Tensor.

OS

The OS library provides functions for file and directory handling in an operating system-independent manner. It is used in this project to verify file paths, check the existence of event log files, manage directories, and handle file-related operations safely. This ensures reliable access to input datasets and stored models during execution.

CSV

The CSV module is used for event log storage and retrieval in a structured text format. Mouse and keyboard interaction data are stored in CSV files, making them easy to read, write, and process. In this project, the CSV format serves as a lightweight and efficient medium for logging user interaction events and storing extracted features for further analysis.

Streamlit

Streamlit is an open-source Python library designed to easily create and share custom web applications for machine learning and data science. In this project, it is used to build the interactive frontend user interface. It allows users to start their webcam, view the live video feed, and receive real-time visual feedback and posture corrections directly within their web browser without requiring complex backend web development.

Time

The Time module is a standard Python library that provides various time-related functions and is crucial for monitoring system performance during live video processing. It is primarily used to calculate the Frames Per Second (FPS). By measuring the exact time elapsed between processing the current video frame and the previous one, the application can display the FPS on the screen, verifying that the pose detection pipeline is operating in real-time without significant latency.

Keras

Keras is a high-level neural networks API, typically running on top of TensorFlow, designed for fast experimentation and deep learning deployment. Specifically, the `load_model` function is utilized to load the pre-trained deep learning model from the local disk into the application's memory. This allows the system to bypass the training phase and instantly begin making predictions and classifying incorrect postures based on the live data stream.

Subprocess

The Subprocess module allows the Python application to spawn new background processes, execute terminal commands, and interact with the operating system's command line. In interactive web applications, it is often utilized to manage the execution of external Python scripts, handle system-level tasks, or facilitate the automated launching and restarting of the Streamlit server environment directly from the code.

Sys

The Sys module provides access to variables and functions that interact directly with the Python interpreter. It is used for system-level operations within the script. This includes manipulating the Python path to ensure all custom project modules and directories are located correctly during execution, or executing a clean, graceful exit of the application when the user terminates the live webcam feed.

3.4 DEEP LEARNING

Deep learning, a highly specialized subset of machine learning, employs artificial neural networks inspired by the human brain to learn from vast amounts of data. It creates multi-layered algorithms capable of detecting complex, non-linear patterns without requiring explicit manual feature engineering. Widely applied in computer vision, natural language processing, and sequence prediction, deep learning is essential for analyzing dynamic continuous data. By learning from large datasets of structured spatial and temporal data, it builds robust models that can evaluate new, real-time video inputs with exceptional precision.

3.4.1 Long Short-Term Memory (LSTM) Network

Long Short-Term Memory (LSTM) is an advanced architecture of Recurrent Neural Networks (RNNs) specifically designed to learn and remember long-term dependencies in sequential data. Unlike standard feedforward neural networks that process individual inputs in isolation, LSTMs utilize internal memory mechanisms to retain context from previous inputs. This makes them highly effective for time-series analysis and video processing. In this project, a pre-trained LSTM network is utilized to classify real-time human posture as correct or incorrect by analyzing a continuous sequence of 3D skeletal landmarks extracted from consecutive video frames.

Why do we use an LSTM Network?

An LSTM network is selected for this project due to its exceptional capability in handling sequential temporal data. Human movement and posture are not static; they are continuous actions over time. Analyzing a sequence of frames rather than a single isolated image provides a much more accurate representation of the user's actual pose.

The key reasons for using an LSTM Network are:

- It excels at processing sequential data and time-series inputs from live video.
- It captures the temporal dynamics and transitional movements of the human body.
- It overcomes the "vanishing gradient" problem of traditional RNNs, allowing it to remember past movements effectively.
- It significantly reduces flickering and false classifications by considering the context of previous frames.
- It handles high-dimensional spatial data (like 33 specific 3D landmarks) continuously.
- It is highly optimized for real-time temporal inference using frameworks like TensorFlow and Keras.

How does an LSTM Network work?

The LSTM network operates by passing sequential input data through specialized internal structures called "cells." Each cell contains three distinct gates the Forget Gate, the Input Gate, and the Output Gate which regulate the flow of information, deciding what past data to remember and what new data to add.

When live video frames are processed during real-time execution:

- MediaPipe extracts the 3D structural coordinates from the user across a continuous window of consecutive frames (e.g., the last 10-30 frames).
- This sequential, time-stamped numerical data is fed into the input layer of the loaded LSTM network.
- The data passes through the LSTM cells, where the gates continuously update the network's hidden state, merging the spatial coordinates of the current frame with the memory of the previous frames.

- The final output layer processes this rich temporal context to produce a highly stable classification score, determining if the ongoing pose requires correction.

Advantages of LSTM Networks:

The major advantages of using an LSTM Network in this project include:

- Superior accuracy for continuous movements compared to static frame-by-frame analysis
- Smooth and stable real-time predictions, minimizing jitter in the visual feedback interface
- Robustness against temporary occlusions or momentarily missing landmark data in a single frame
- Ability to maintain contextual memory of the user's previous poses to anticipate posture degradation
- Scalability to handle complex, continuous data streams from standard webcams smoothly

CHAPTER 4

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

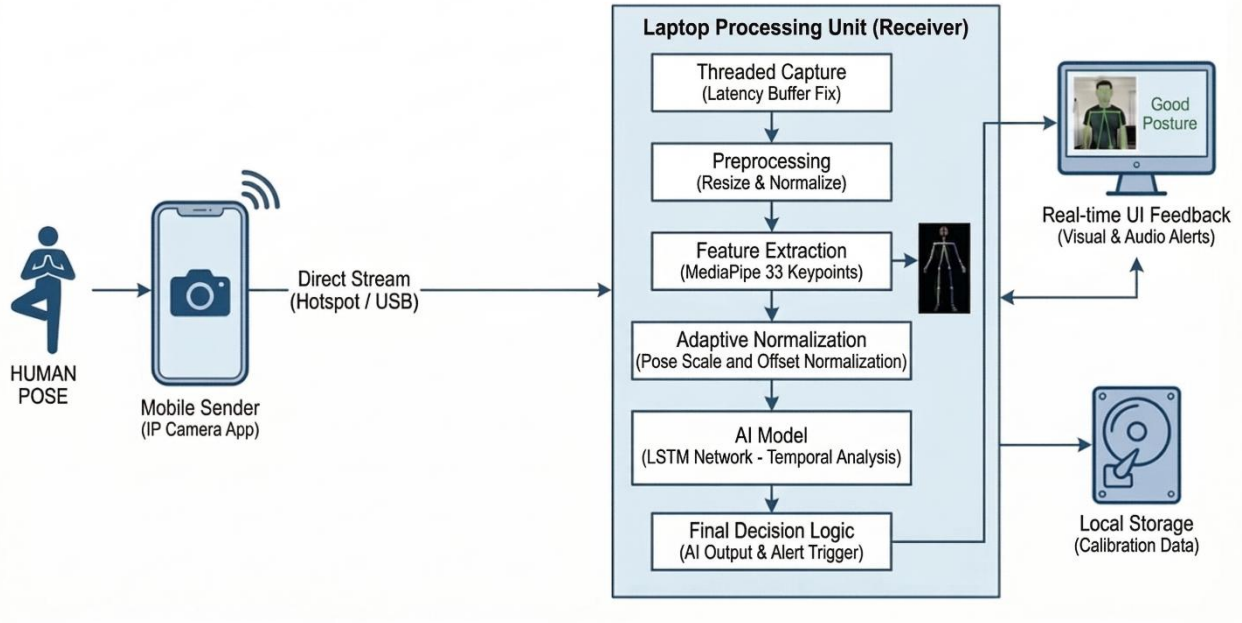


Figure 4.1: System Architecture

The proposed architecture for the Real-Time Human Pose Detection and Correction system operates as a continuous, low-latency pipeline divided into three primary stages: data acquisition, core processing, and output generation. To ensure high-quality video input without relying on a standard, low-resolution laptop webcam, the system utilizes a smartphone acting as an external IP camera. This mobile sender captures the live video feed of the user's posture and transmits it directly to the laptop processing unit via a dedicated Wi-Fi hotspot or a wired USB connection. This direct streaming approach is crucial as it eliminates local network lag and ensures a highly stable, real-time data flow into the processing pipeline.

Once the laptop receiver acquires the video stream, it initiates a multi-step computational process. It begins with a threaded capture mechanism designed to fix standard latency buffers by constantly pulling only the absolute newest frame from the camera, ensuring the system evaluates the user's present state rather than a delayed

image. These raw frames then undergo preprocessing, where they are resized and color-normalized to optimize processing speed. Following this, the frames are fed into the MediaPipe Pose framework for feature extraction, which accurately identifies and plots 33 specific 3D skeletal keypoints across the user's body. Because users vary significantly in height and sitting distance from the camera, these raw spatial coordinates are passed through an adaptive normalization module. This step scales and adjusts the keypoints based on baseline calibration data, ensuring the artificial intelligence evaluates the fundamental proportions and angles of the pose rather than raw pixel distances.

This standardized coordinate data is subsequently fed into the core AI model: a Long Short-Term Memory (LSTM) network. Because human posture is a continuous physical action rather than a static pose, the LSTM performs temporal analysis on a sequence of recent frames, allowing it to understand the dynamic flow of the user's movements and accurately classify the current posture state. The output from the LSTM model is then evaluated by the final decision logic module, which determines if the pose has degraded into a poor position based on predefined thresholds. Based on this final determination, the system simultaneously drives two distinct outputs. First, it updates the real-time user interface to provide immediate visual and audio feedback, displaying warnings directly over the live video feed if correction is needed. Concurrently, it interacts with local storage to securely save and retrieve the user's calibration data, ensuring a seamless experience where the user does not need to repeatedly recalibrate their body metrics during every session.

4.2 DATA FLOW DIAGRAM

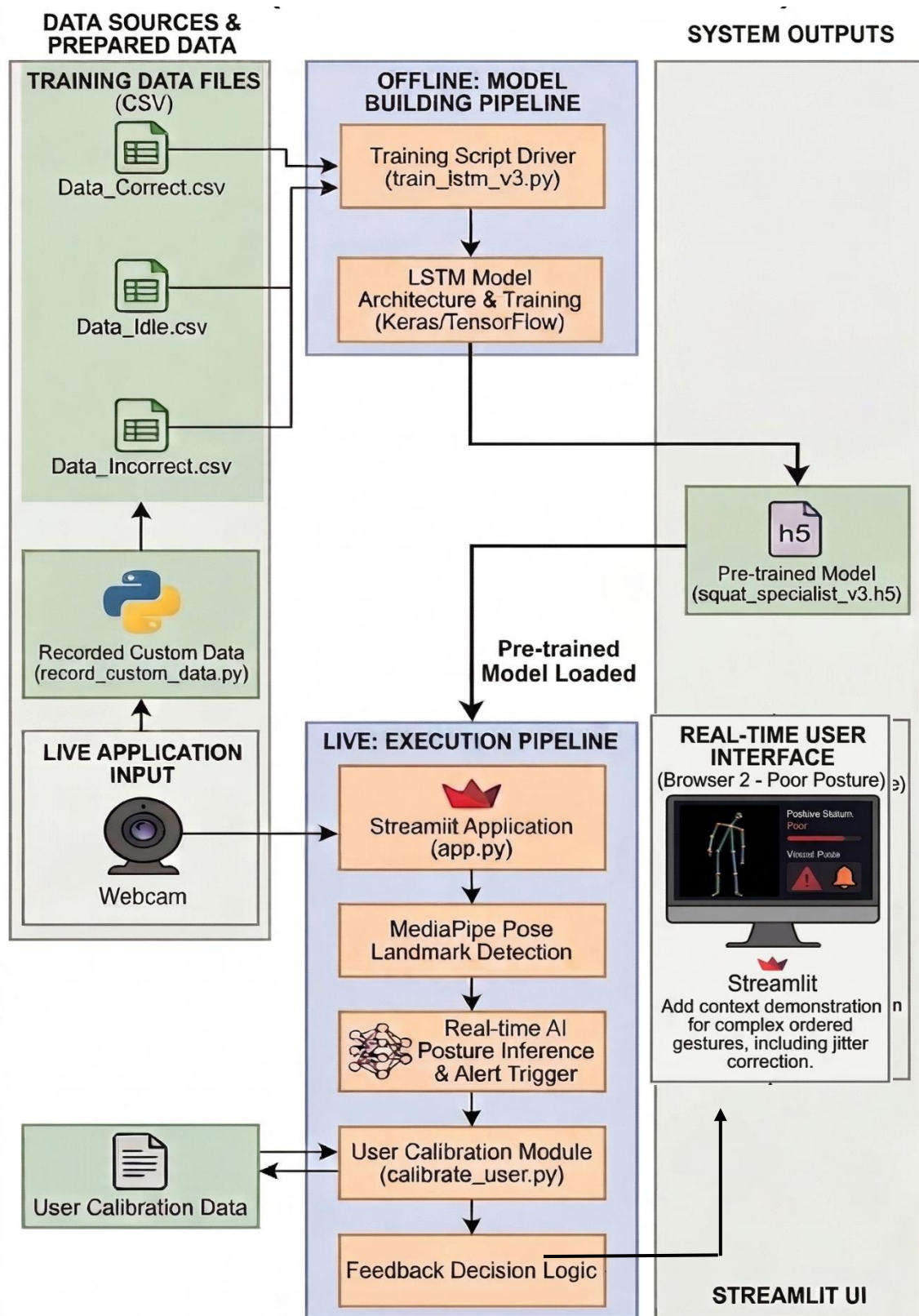


Figure 4.2: Human Pose Correction System Data Flow Diagram

The data flow begins with the Data Sources and Prepared Data layer, where raw movement information is collected to distinguish between various physical states. During the offline phase, specific body movements are categorized into three primary CSV datasets `Data_Correct.csv`, `Data_Idle.csv`, and `Data_Incorrect.csv` which are typically generated using the `record_custom_data.py` script. For the live application, the system ingests high-frequency visual input from a Webcam, while a User Calibration Data module captures personalized baseline metrics to account for individual body proportions. This information is then funneled into the Offline Model Building Pipeline, where the `train_lstm_v3.py` script manages the ingestion of data into a Long Short-Term Memory (LSTM) neural network. Developed using Keras and TensorFlow, this architecture recognizes temporal patterns in movement, eventually exporting the intelligence as a Pre-trained Model named `squat_specialist_v3.h5`.

As the system transitions into the Live Execution Pipeline, the `squat_specialist_v3.h5` model is integrated into a web-based dashboard. The user interface is initialized by executing the command `streamlit run app.py`, which launches the interactive environment for active monitoring. As the live webcam feed enters the system, MediaPipe Pose Landmark Detection identifies 33 key skeletal points on the user's body in real-time. These coordinates are processed through the Real-time AI Posture Inference engine, which compares the live movement against the patterns learned during training. The `calibrate_user.py` script then evaluates any deviations, incorporating the user's calibration data to ensure that alerts are accurate and not triggered by natural body variance. Finally, the process concludes at the System Outputs within the Streamlit UI, where the user receives immediate visual feedback, including live posture status and skeletal overlays for physical form correction.

The technical integrity of the system relies on the seamless transition of high-dimensional coordinate data from the MediaPipe framework into the temporal analysis layers of the LSTM. By utilizing a windowed approach to data processing, the inference engine evaluates sequences of skeletal landmarks to calculate joint angles and displacement vectors, comparing them against the statistically optimal thresholds defined during the training epoch. This ensures that the Feedback Decision Logic can distinguish between minor user adjustments and significant form failures. The final output on the Streamlit UI serves as a low-latency monitoring dashboard, rendering real-time pose overlays and probabilistic confidence scores. This end-to-end architecture demonstrates a robust integration of computer vision and deep learning, providing a scalable solution for automated biomechanical assessment and user-specific ergonomic intervention.

CHAPTER 5

PROJECT DESCRIPTION

5.1 MODULES

The list of the seven modules for our proposed system:

- Data Acquisition Module
- Pose Estimation Module
- Feature Extraction Module
- Temporal Filter Module
- Calibration Module
- Final Decision Module
- Feedback Generation Module

5.2 MODULE DESCRIPTION

5.2.1 Data Acquisition Module

This module establishes the high-speed hardware interface between the Mobile Sender (IP Camera App) and the Laptop Processing Unit via a Direct Stream (Hotspot or USB). To ensure real-time performance, it implements a Threaded Capture mechanism with a latency buffer fix to extract frames at a consistent rate. Immediately following capture, the module performs critical Preprocessing steps it executes an RGB conversion to transform frames from the default BGR color space to the RGB format required for inference, and applies Resize and Normalize operations to standardize the data regardless of the mobile device's native resolution or lighting conditions.

5.2.2 Pose Estimation Module

The system utilizes the MediaPipe BlazePose algorithm to transform the preprocessed pixel data into a structured digital skeleton. This lightweight, optimized model identifies 33 skeletal keypoints in 3D space for every frame. By capturing x, y, and z(depth) coordinates for each landmark, this module provides the foundational spatial data required for complex action recognition.

5.2.3 Feature Extraction Module

In this module, raw coordinates are converted into a comprehensive 99-value feature vector (x, y, z coordinates for all 33 keypoints). Unlike static angle calculations, this approach preserves the full spatial context of the user's posture. These feature vectors are then prepared for sequential analysis, representing the user's movement trajectory independently of their specific position in the camera frame.

5.2.4 Temporal Analysis Module

The core intelligence of the system resides in this module, which utilizes a Long Short-Term Memory (LSTM) Network. The model is specifically trained on the UI-PRMD dataset to recognize sequences of motion over time. By analyzing a sliding window of 30 frames, the LSTM can distinguish between various physical exercises and determine if the temporal pattern of the movement matches the "ideal" rehabilitation form.

5.2.5 Calibration Module

To account for diverse body types and varying camera distances, this module implements an Adaptive Normalization phase. During a brief initial calibration, the system scales the skeletal data to a standardized range. This ensures that the AI model's predictions and the subsequent error detection remain accurate regardless of the user's physical stature or the mobile device's placement.

5.2.6 Final Decision Module

This module evaluates the AI Model's output against a Dynamic Threshold established during calibration. If the current pose sequence deviates significantly from the target action, the diagnostic logic isolates the specific error. By comparing real-time temporal data against learned weights, it determines the exact moment and joint location where the form is incorrect.

5.2.7 Feedback Generation Module

This module translates technical diagnostics into actionable Real-time UI Feedback. It renders a skeleton overlay and provides both visual and audio alerts (e.g., "Good Posture" or specific correction prompts) directly to the user. Simultaneously, all session data including CSV coordinate logs and video recordings is saved to Local Storage for future review and progress tracking.

CHAPTER 6

SYSTEM IMPLEMENTATION

The system is built on a Python framework using OpenCV for real-time video processing. Core analysis utilizes MediaPipe for pose estimation, converting skeletal landmarks into geometric feature vectors via NumPy. These vectors feed into a Long Short-Term Memory (LSTM) network for sequence classification, enabling the system to render immediate, corrective feedback.

6.1 DATA ACQUISITION (Video Capture)

This module establishes the high-speed hardware interface between the Mobile Sender (IP Camera App) and the Laptop Processing Unit via a Direct Stream (Hotspot or USB). To ensure real-time performance, it implements a Threaded Capture mechanism with a latency buffer fix to extract frames at a consistent rate.

Process Flow

- Stream Initialization: The system initializes the default video capture device (Index 0) using the OpenCV library.
- Frame Capture: The module captures video frames at a target rate of 30 frames per second (FPS). Each frame is read as a matrix of pixels in BGR (Blue-Green-Red) color space.
- Preprocessing:
 - Color Conversion: The frame is converted from BGR to RGB (Red-Green-Blue) format, as the downstream Pose Estimation model requires RGB input.
 - Resizing: If necessary, frames are resized to a standard resolution (e.g., 640x480) to optimize processing speed without sacrificing accuracy.

Module Implementation for Video Capture and Preprocessing

```
import cv2
class VideoSource:
    def __init__(self, source_index=0, width=640, height=480):
        self.cap = cv2.VideoCapture(source_index)
        self.cap.set(3, width)
        self.cap.set(4, height)
        if not self.cap.isOpened():
            print("Error: Could not access the")
    def get_frame(self):
        success, frame = self.cap.read() if
        success:
            frame = cv2.flip(frame, 1)
        return success, frame
```

6.2 POSE ESTIMATION

The Pose Estimation module is responsible for identifying the human skeletal structure within the video frames. This project utilizes Google MediaPipe Pose, a state-of-the-art machine learning solution that employs a BlazePose Convolutional Neural Network (CNN).

Process Flow

- Instead of training a vision model from scratch, this module leverages Transfer Learning. The MediaPipe model infers a 33-point skeletal topology from a single 2D image.
- Face Detection: The model first locates the user's face to establish a region of interest.
- Key point Regression: It then predicts the coordinates of 33 specific body landmarks (e.g., nose, shoulders, hips, knees, ankles).
- 3D Projection: The model estimates a 3D coordinate space (x, y, z) for each landmark, where z represents the depth relative to the hips.

Module Implementation for Pose Estimation

```
import cv2
import mediapipe as mp
def find_body(self, frame):
    img_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = self.pose.process(img_rgb)
    idf results.pose_landmarks:
        self.mp_draw.draw_landmarks( frame, results.pose_landmarks,
        self.mp_pose.POSE_CONNECTIONS)
    return frame, results.pose_landmarks
```

6.3 FEATURE EXTRACTION

The Feature Extraction module functions as the system's data serialization interface. Its primary objective is to capture the raw skeletal output generated by the Pose Estimation layer and persistently store it into a structured dataset. Rather than processing or modifying the data, this module acts as a “logger,” preserving the exact geometric state of the user's movement frame-by-frame for future analysis or model training.

Raw Data Extraction

The module interfaces directly with the output of the Pose Estimation layer (MediaPipe). For every video frame processed, it retrieves the raw data for the 33 skeletal landmarks.

Each landmark provides four raw attributes:

- **x:** Horizontal coordinate (Normalized 0.0 to 1.0).
- **y:** Vertical coordinate (Normalized 0.0 to 1.0).
- **z:** Depth coordinate (Relative to the hip center).
- **v:** Visibility score (0.0 to 1.0).

Module Implementation for Feature Extraction

```
import csv
import time

class DataLogger:
    def __init__(self, filename="raw_pose_data.csv"):
        self.file = open(filename, mode='w', newline='')
        self.writer = csv.writer(self.file)
        header = ['timestamp']
        for i in range(33):
            header.extend([f'lm_{i}_x', f'lm_{i}_y', f'lm_{i}_z', f'lm_{i}_v'])
        self.writer.writerow(header)
        print(f"Logging started: {filename}")
```

6.4 TEMPORAL FILTER

The Temporal Filter Module is responsible for data stabilization and windowing. Because raw pose estimation can produce “shaky” landmarks due to camera noise or lighting changes, this module implements a Moving Average Filter to smooth the coordinates. Furthermore, it transforms the data into a Sliding Window format, ensuring the LSTM receives a continuous stream of the last 30 frames for every prediction cycle.

Process Flow

- **Coordinate Smoothing:** The module applies a weighted average to the x, y, z coordinates of all 33 landmarks to prevent the UI from flickering between states.
- **Windowing (Sequence Creation):** It maintains a FIFO (First-In-First-Out) buffer that collects 30 consecutive frames. When a new frame enters, the oldest frame is discarded, creating a “sliding” effect.
- **Dimensionality Reshaping:** The 1D feature vector of 240 elements is reshaped into a 3D tensor of shape (1, 30, 240) to match the input requirements of the LSTM model.

Module Implementation for Temporal Stabilization and Sliding Window Logic

```
import numpy as np
```

```
class TemporalFilter:
```

```
    def __init__(self, window_size=30, feature_count=240):
```

```
        self.window_size = window_size
```

```
        self.feature_count = feature_count
```

```
        self.buffer = []
```

```
    def smooth_and_buffer(self, new_features)
```

```
        self.buffer.append(new_features)
```

```
        if len(self.buffer) > self.window_size:
```

```
            self.buffer.pop(0)
```

```
        if len(self.buffer) == self.window_size:
```

```
            return np.expand_dims(np.array(self.buffer), axis=0)
```

```
        return None
```

```
    def apply_moving_average(self, data_stream, span=5):
```

```
        if len(self.buffer) < span:
```

```
            return data_stream
```

```
        return np.mean(self.buffer[-span:], axis=0)
```

6.5 CALIBRATION

The Calibration Module is the personalization engine of the system. Its primary function is to define a “User-Specific Goal” based on the individual’s current physical capability rather than a generic fitness standard. This is critical for clinical rehabilitation, where a patient may have a restricted range of motion (ROM) due to surgery or chronic pain.

Process Flow

- **Baseline Detection:** The system prompts the user to perform their maximum “safe” squat depth.
- **Angle Capture:** The module calculates the real-time average knee angle across

both legs using the geometric keypoints (Hip-Knee-Ankle).

- **Data Persistence:** Once the user triggers the “Save” command (e.g., pressing the ‘S’ key), the module writes this specific angle value into a local configuration file (user_calibration.txt).
- **Integration:** The Live Inference module reads this file upon startup, automatically shifting the “PERFECT SQUAT” threshold to match the user’s calibrated limit.

Module Implementation for Calibration and Data Persistence

```
import cv2
import os

class CalibrationManager:
    def __init__(self, file_path="user_calibration.txt"):
        self.file_path = file_path
        self.calibrated_angle = 120.0 # Default standard depth

    def save_calibration(self, angle):
        try:
            with open(self.file_path, "w") as f:
                f.write(str(round(angle, 2)))
            print(f"Target saved: {int(angle)}°")
            return True
        except Exception as e:
            print(f"Error saving calibration: {e}")
            return False

    def load_calibration(self):
        if os.path.exists(self.file_path):
            with open(self.file_path, "r") as f:
                self.calibrated_angle = float(f.read().strip())
        return self.calibrated_angle
```

6.6 FINAL DECISION

The Final Decision Module serves as the system's analytical engine, responsible for interpreting the probabilistic output of the LSTM network. It compares the real-time inference scores against the Dynamic Threshold established in the Calibration Module. By analyzing the weighted influence of specific skeletal landmarks within the current 30-frame window, this module isolates the exact biomechanical deviation (e.g., knee collapse or shallow depth) and determines if the movement constitutes a "Correct," "Incorrect," or "Idle" state.

Process Flow

- Inference Evaluation: The module receives a Softmax probability array from the LSTM (e.g., [0.85, 0.10, 0.05]).
- Threshold Comparison: It checks if the "Correct" class probability exceeds the user-specific threshold loaded from `user_calibration.txt`.
- Error Isolation: If the "Incorrect" probability dominates, the module cross-references the current knee angles against the calibrated baseline to pinpoint the specific joint location of the form failure.
- State Locking: To prevent "flickering" feedback, the module requires a consistent state for at least 3 consecutive frames before updating the system status.

Module Implementation for Final Decision

```
import numpy as np
class DecisionModule:
    def __init__(self, calibration_manager):
        self.threshold = calibration_manager.load_calibration()
        self.current_state = "IDLE"
    def analyze_output(self, prediction, current_angles):
prob_correct = prediction[0]
prob_incorrect = prediction[1]
if prob_correct > 0.75: # High confidence correct
```

```

        self.current_state = "GOOD POSTURE"
        diagnostic = "Target Depth Achieved"
    elif prob_incorrect > 0.40:
        self.current_state = "POOR POSTURE"
        if current_angles['knee'] > self.threshold + 10:
            diagnostic = "Error: Squat Depth Too Shallow"
        else:
            diagnostic = "Error: Knee Alignment Instability"
    else:
        self.current_state = "IDLE"
        diagnostic = "Waiting for Movement..."
    return self.current_state, diagnostic

```

6.7 FEEDBACK GENERATION

The Feedback Generation Module is the primary communication layer between the processing core and the user. It translates technical diagnostic data into actionable, multi-modal cues delivered through the Streamlit interface. This module is responsible for rendering the augmented reality (AR) skeleton overlay, triggering synchronized audio-visual alerts, and maintaining the data persistence layer by logging all processed metrics to local storage for long-term progress tracking.

Process Flow

- **UI Rendering:** Utilizing the streamlit run app.py environment, it draws a 2D skeleton wireframe over the video feed, dynamically changing joint colors (e.g., Green for alignment, Red for deviation).
- **Alert Triggering:** Based on the Decision Module's state, it displays high-contrast text prompts (e.g., "SQUAT LOWER") and plays localized audio cues to guide the user in real-time.

Module Implementation for Real-Time UI Feedback Generation

```
import cv2
import pandas as pd
import datetime

class FeedbackGenerator:
    def __init__(self, log_file="session_summary.csv"):
        self.log_file = log_file

    def generate_ui(self, frame, state, diagnostic):
        color = (0, 255, 0) if state == "GOOD POSTURE" else (0, 0, 255)
        if state == "IDLE": color = (255, 255, 255)
        cv2.rectangle(frame, (0, 0), (350, 100), (40, 40, 40), -1)
        cv2.putText(frame, f"STATUS: {state}", (20, 40),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, color, 2)
        cv2.putText(frame, diagnostic, (20, 80),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 1)
        return frame

    def log_session(self, state, landmarks):
        log_entry = {
            "timestamp": [datetime.datetime.now().strftime("%H:%M:%S.%f")],
            "state": [state],
            "landmarks_payload": [landmarks]}
        pd.DataFrame(log_entry).to_csv(self.log_file,
            mode='a',header=False,index=False)
```


CHAPTER 7

SYSTEM TESTING

7.1 TESTING

Testing is the strategic process of executing the Human Pose Correction System with the explicit intent of uncovering logical errors, synchronization issues, or biomechanical inaccuracies. A successful test case is one that identifies a previously undiscovered flaw in the landmark detection or sequence classification pipeline. System testing represents the final stage of implementation, ensuring that the integration between the webcam feed, the MediaPipe inference engine, and the LSTM predictive model functions accurately and efficiently before real-time deployment.

This process serves as the ultimate quality assurance review of the system's specifications, design, and algorithmic coding. By simulating various user movements and environmental conditions, testing verifies that the entire suite of Python scripts from data acquisition to feedback generation operates as a cohesive unit. This is the critical final opportunity to refine the Dynamic Threshold and ensure the Streamlit UI provides stable, low-latency guidance to the user.

7.2 DIFFERENT LEVELS OF TESTING

- Unit Testing
- Integration Testing
- Functional Testing
- Model Performance Testing

7.2.1 Unit Testing

Unit testing involves the isolated validation of each individual module within the Pose Correction System to ensure they perform their intended mathematical and logical functions correctly. Each Python component is tested independently to verify its reliability under diverse input conditions.

The Video Source module is tested to ensure it correctly initializes the hardware interface and performs the necessary BGR-to-RGB color conversions at a consistent 30 FPS. The Skeletal Topology Extraction module (MediaPipe) is validated to confirm it accurately identifies all 33 landmarks even under varying lighting conditions. The DataLogger (record_custom_data.py) is tested to ensure that geometric features (x, y, z, v) are serialized into CSV format without data loss. Finally, the Temporal Filter is unit-tested to verify that the sliding window logic correctly maintains a FIFO buffer of 30 frames and reshapes the data into the required 3D tensor (1, 30, 240) for the LSTM.

7.2.2 Integration Testing

Integration testing focuses on the data flow between modules to ensure that the entire pipeline functions correctly when components are combined. This phase verifies that the high-dimensional coordinate data produced by MediaPipe is successfully ingested by the Temporal Filter without causing memory leaks or latency spikes. A key focus of integration testing is the connection between the Final Decision Module and the Calibration Manager. The system is tested to ensure that the user_calibration.txt file is correctly read upon startup and that its values successfully shift the prediction thresholds within the inference engine. Furthermore, the integration between the LSTM model (squat_specialist_v3.h5) and the Feedback Generation Module is validated to ensure that a "Poor Posture" prediction immediately triggers the corresponding visual and audio alerts on the Streamlit dashboard.

7.2.3 Functional Testing

Functional testing ensures the system satisfies all project objectives and user requirements in a real-world scenario. The system is tested to verify that it can continuously monitor a user's movement and provide instantaneous feedback. Tests are conducted to confirm that the Streamlit UI correctly renders the colored skeleton wireframe and that the "Correction Prompts" appear only when the biomechanical deviation exceeds the established threshold. The Local Storage system is also verified to ensure that at the end of a session, both the coordinate logs and the summary reports are accurately saved for the user to review. Functional testing confirms that the system operates reliably as a real-time ergonomic coach.

7.2.4 Model Performance Testing

Model performance testing evaluates the accuracy of the LSTM network in classifying movement sequences. After training the model on the prepared datasets, it is challenged with a "Test Set" of unseen movements to measure its predictive power. The system utilizes specific metrics to evaluate the AI's performance:

- Accuracy: The percentage of total poses correctly identified as Correct, Incorrect, or Idle.
- Precision and Recall: Used to ensure the system does not miss "Incorrect" poses (High Recall) while avoiding "False Alarms" for users with good form (High Precision).
- Latency (ms): The time taken from frame capture to UI feedback, ensuring the system maintains a real-time feel (<100ms).

CHAPTER 8

CONCLUSION

The proposed real-time human pose correction system represents a significant advancement over traditional fitness and rehabilitation monitoring tools by addressing their critical limitations, such as offline processing, delayed feedback, and cognitive information overload. To overcome these issues, the project successfully deploys a synchronous video-stream pipeline integrated with a Long Short-Term Memory (LSTM) network for deep temporal analysis. By continuously tracking 33 distinct 3D skeletal landmarks across a sliding window of consecutive frames, the system's diagnostic logic synthesizes sequential biomechanical data and utilizes user-calibrated dynamic thresholds to effectively filter out transient sensor noise, identifying only genuine, persistent postural errors. Rather than overwhelming the user with excessive instructions, the framework prioritizes critical misalignments and delivers simplified, multimodal feedback through visual overlays and low-latency audio cues. Ultimately, this highly adaptable solution empowers users to make instantaneous, proprioceptive corrections during dynamic movements, thereby maximizing learning effectiveness, enhancing kinematic accuracy, and significantly reducing the risk of injury across clinical rehabilitation, yoga practice, and general fitness applications.

APPENDICES

APPENDIX1: SOURCE CODE

1. record_custom_data.py

```
import cv2
import numpy as np
import mediapipe as mp
import pandas as pd
import time
import os

EXPECTED_FEATURES = 240

def calculate_angle(a, b, c):
    a = np.array([a.x, a.y])
    b = np.array([b.x, b.y])
    c = np.array([c.x, c.y])
    radians = np.arctan2(c[1]-b[1], c[0]-b[0]) - np.arctan2(a[1]-b[1], a[0]-b[0])
    angle = np.abs(radians*180.0/np.pi)
    return 360 - angle if angle > 180.0 else angle

def main():
    print("=====")
    print("  POSEGURU: HYBRID DATA RECORDER  ")
    print("=====")
    print("0 - Record CORRECT Squats")
    print("1 - Record INCORRECT Squats")
    print("2 - Record IDLE (Standing still)")
    label = input("Enter the number for what you are recording: ")
    if label == '0':
        filename = "Data_Correct.csv"
    elif label == '1':
```

```

    filename = "Data_Incorrect.csv"
elif label == '2':
    filename = "Data_Idle.csv"
else:
    print("Invalid input!")
    return

mp_pose = mp.solutions.pose
pose = mp_pose.Pose()
mp_draw = mp.solutions.drawing_utils
cap = cv2.VideoCapture(0)
data_list = []
print("Get ready! Recording starts in 5 seconds...")
cv2.waitKey(5000)
print("RECORDING NOW! (Do the movement for 30 seconds)")
start_time = time.time()
while time.time() - start_time < 30:
    success, frame = cap.read()
    if not success:
        continue
    frame = cv2.flip(frame, 1)
    results = pose.process(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
    if results.pose_landmarks:
        lm = results.pose_landmarks.landmark
        features = []
        for landmark in lm:
            features.extend([0.0] * (EXPECTED_FEATURES - len(features)))
        data_list.append(features)
    cv2.imshow("Recording Data", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):

```

```

        break
cap.release()
cv2.destroyAllWindows()
df = pd.DataFrame(data_list)
if os.path.exists(filename):
    df.to_csv(filename, mode='a', header=False, index=False)
else:
    df.to_csv(filename, index=False)
print(f'Successfully {len(data_list)} frames to {filename} with Angle Data!')
if __name__ == "__main__":
    main()

```

2. train_lstm_v3.py

```

import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, LSTM, Dense, Dropout, concatenate,
Lambda
from tensorflow.keras.utils import to_categorical
from sklearn.model_selection import train_test_split
SEQUENCE_LENGTH = 30
EXPECTED_FEATURES = 240
MODEL_NAME = "squat_specialist_v3.h5"
def load_data():
    print("Loading CSV Data...")
    df_correct = pd.read_csv("Data_Correct.csv").values
    df_incorrect = pd.read_csv("Data_Incorrect.csv").values
    df_idle = pd.read_csv("Data_Idle.csv").values
    for i in range(0, len(df_idle) - SEQUENCE_LENGTH)

```

```

        sequences.append(df_idle[i : i + SEQUENCE_LENGTH])
        labels.append(2)

    return np.array(sequences), np.array(labels)

def get_branch_features(x, landmark_indices):
    feature_indices = []
    for i in landmark_indices:
        feature_indices.extend([i*4, i*4+1, i*4+2, i*4+3])
    return tf.gather(x, feature_indices, axis=2)

def get_angle_features(x):
    return tf.gather(x, [132, 133], axis=2)

def build_spatio_temporal_model():
    inputs = Input(shape=(SEQUENCE_LENGTH, EXPECTED_FEATURES))
    idx_torso = [0,1,2,3,4,5,6,7,8,9,10, 11,12, 23,24]
    idx_arms = [13, 14, 15, 16, 17, 18, 19, 20, 21, 22]
    idx_legs = [25, 26, 27, 28, 29, 30, 31, 32]
    branch_torso = Lambda(lambda x: get_branch_features(x, idx_torso))
    branch_arms = Lambda(lambda x: get_branch_features(x, idx_arms))
    branch_legs = Lambda(lambda x: get_branch_features(x, idx_legs))
    branch_angles = Lambda(get_angle_features, name="Angles_Split")(inputs)
    lstm_torso = LSTM(32, return_sequences=False)

    dense_1 = Dense(128, activation='relu')(merged)
    drop_1 = Dropout(0.3)(dense_1)
    dense_2 = Dense(64, activation='relu')(drop_1)
    outputs = Dense(3, activation='softmax', name="Final_Decision")(dense_2)
    model = Model(inputs=inputs, outputs=outputs)
    return model

def main():
    X, y = load_data()

```



```

y_hot = to_categorical(y, num_classes=3)
X_train, X_test, y_train, y_test = train_test_split(X, y_hot, test_size=0.2)
print("\nBuilding 4-Branch Hybrid Spatio-Temporal Architecture...")
model = build_spatio_temporal_model()
model.summary()
print("\nTraining Advanced Model...")
model.fit(X_train, y_train, epochs=40, batch_size=16)
model.save(MODEL_NAME)
print(f"\nMASTERPIECE Your new angle-aware {MODEL_NAME}")
if __name__ == "__main__":
    main()

```

3. calibrate_user.py

```

import cv2
import mediapipe as mp
import numpy as np
import time

CAMERA_SOURCE = "https://192.168.1.122:8080/video"

def calculate_angle(a, b, c):
    a = np.array([a.x, a.y])
    b = np.array([b.x, b.y])
    c = np.array([c.x, c.y])
    radians = np.arctan2(c[1]-b[1], c[0]-b[0]) - np.arctan2(a[1]-b[1], a[0]-b[0])
    angle = np.abs(radians * 180.0 / np.pi)
    if angle > 180.0:
        angle = 360 - angle
    return angle

mp_pose = mp.solutions.pose
mp_draw = mp.solutions.drawing_utils
cap = cv2.VideoCapture(CAMERA_SOURCE)

```

```

if not cap.isOpened():
    print("=====")
    print(f"ERROR: Could not connect to the camera at:")
    print(f" {CAMERA_SOURCE}")
    print(" Make sure the IP Webcam app is running on your phone!")
    print("=====")
    time.sleep(5)
    exit()
print("Starting Calibration...")
start_time = time.time()
prep_duration = 10
cal_duration = 10
lowest_angle = 180.0
mode = "PREP"
while cap.isOpened():
    success, frame = cap.read()
    if not success:
        break
    frame = cv2.flip(frame, 1)
    results = pose.process(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
    current_time = time.time()
    elapsed = int(current_time - start_time)
    if results.pose_landmarks:
        lm = results.pose_landmarks.landmark
        knee_angle = (calculate_angle(lm[23], lm[25], lm[27]))
    if mode == "PREP":
        time_left = prep_duration - elapsed
        if time_left <= 0:
            mode = "CALIBRATE"

```

```

        start_time = time.time()
    elif mode == "CALIBRATE":
        time_left = cal_duration - int(current_time - start_time)
        if 'knee_angle' in locals() and knee_angle < lowest_angle and knee_angle :
            lowest_angle = knee_angle
        if time_left <= 0:
            break

    cv2.imshow('Calibration Window', frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
with open("user_calibration.txt", "w") as f:
    f.write(str(lowest_angle))
print(f"Calibration Complete! New target saved: {lowest_angle}")

```

4. app.py

```

import streamlit as st
import cv2
import numpy as np
import mediapipe as mp
import os
import time
import tensorflow as tf
from tensorflow.keras.models import load_model
import subprocess
import sys

CAMERA_SOURCE = "https://192.168.1.122:8080/video"

def speak(text):
    try:

```

```

    script = f"""
import pytsx3
try:
    engine = pytsx3.init()
    engine.setProperty('rate', 170)
    engine.say("{text}")
    engine.runAndWait()
except:
    pass
<style>
.stApp { background-color: #0f172a; color: #f8faff; }
div.stButton > button:first-child {
    background-color: #1e293b; color: #f8faff; border: 2px solid #3b82f6;
    border-radius: 12px; font-size: 18px; font-weight: bold; transition: all 0.3s ease;
}
div.stButton > button:first-child:hover {
    background-color: #3b82f6; color: #ffffff; border-color: #3b82f6;
    box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
}
.metric-card { background-color: #1e293b; padding: 20px; border-radius: 15px;
text-align: center; border: 1px solid #334155; margin-bottom: 15px; }
.metric-value { font-size: 48px; font-weight: bold; color: #3b82f6; }
.goal-value { font-size: 48px; font-weight: bold; color: #10b981; }
.status-good { color: #10b981; font-size: 24px; font-weight: bold; }
.status-warn { color: #ef4444; font-size: 24px; font-weight: bold; }
.status-idle { color: #f59e0b; font-size: 24px; font-weight: bold; }
</style>
""", unsafe_allow_html=True)
def get_branch_features(x, landmark_indices):

```

```

feature_indices = []
for i in landmark_indices:
    feature_indices.extend([i*4, i*4+1, i*4+2, i*4+3])
return tf.gather(x, feature_indices, axis=2)

def get_angle_features(x):
    return tf.gather(x, [132, 133], axis=2)

def calculate_angle(a, b, c):
    a = np.array([a.x, a.y])
    b = np.array([b.x, b.y])
    c = np.array([c.x, c.y])
    radians = np.arctan2(c[1]-b[1], c[0]-b[0]) - np.arctan2(a[1]-b[1], a[0]-b[0])
    angle = np.abs(radians * 180.0 / np.pi)
    return 360 - angle if angle > 180.0 else angle

@st.cache_resource
def load_ai():
    try:
        model = load_model("squat_specialist_v3.h5",
custom_objects={'get_branch_features': get_branch_features, 'get_angle_features':
get_angle_features, 'tf': tf})
        return model
    except Exception:
        return None

model = load_ai()

mp_pose = mp.solutions.pose
pose = mp_pose.Pose(model_complexity=0, smooth_landmarks=True,
min_detection_confidence=0.85, min_tracking_confidence=0.85)

mp_draw = mp.solutions.drawing_utils
st.title("POSEGURU | Clinical Rehab AI")

if 'target_angle' not in st.session_state:

```

```

if os.path.exists("user_calibration.txt"):
    with open("user_calibration.txt", "r") as f:
        st.session_state.target_angle = float(f.read().strip())
else:
    st.session_state.target_angle = 120.0
if 'camera_active' not in st.session_state:
    st.session_state.camera_active = False
if 'cap' not in st.session_state:
    st.session_state.cap = None
col1, col2 = st.columns([2, 1])
with col1:
    st.markdown("### Live Tracking")
    video_placeholder = st.empty()
    status_placeholder = st.empty()
with col2:
    st.markdown("### Clinical Metrics")
    reps_placeholder = st.empty()
    angle_placeholder = st.empty()
    goal_placeholder = st.empty()
    goal_placeholder.markdown(f"""
        <div class="metric-card">
            <p style="color: #94a3b8; letter-spacing: 2px;">ADAPTED GOAL</p>
            <p class="goal-value">{int(st.session_state.target_angle)}°</p>
        </div>
        """, unsafe_allow_html=True)
    start_btn = st.button("START CAMERA", use_container_width=True)
    stop_btn = st.button("STOP CAMERA", use_container_width=True)
    cal_btn = st.button("CALIBRATE SAFE DEPTH", use_container_width=True)
    if start_btn:

```

```

    st.session_state.camera_active = True
    speak("Camera started. Ready when you are.")
if stop_btn:
    st.session_state.camera_active = False
    if st.session_state.cap is not None:
        st.session_state.cap.release()
        st.session_state.cap = None
    speak("Workout paused.")
if cal_btn:
    st.session_state.camera_active = False
    if st.session_state.cap is not None:
        st.session_state.cap.release()
        st.session_state.cap = None
    with st.spinner("Switching cameras... Please wait 2 seconds!"):
        speak("Switching to calibration mode.")
        time.sleep(2)
        subprocess.run([sys.executable, "calibrate_user.py"])
        if os.path.exists("user_calibration.txt"):
            with open("user_calibration.txt", "r") as f:
                new_target = float(f.read().strip())
                st.session_state.target_angle = new_target
                speak(f"Calibration complete. New target is {int(new_target)} degrees.")
    st.rerun()
if st.session_state.camera_active:
    if st.session_state.cap is None or not st.session_state.cap.isOpened():
        st.session_state.cap = cv2.VideoCapture(CAMERA_SOURCE)
        st.session_state.cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)
    sequence_buffer = []
    squat_state = "up"

```

```

reps = 0
frame_count = 0
st.session_state.current_knee_angle = 180.0
last_spoken_status = ""
last_spoken_time = 0
while st.session_state.cap is not None and st.session_state.cap.isOpened() and
st.session_state.camera_active:
    success, frame = st.session_state.cap.read()
    if not success:
        break
    frame = cv2.flip(frame, 1)
    frame_count += 1
    if frame_count % 2 != 0:
        continue
    results = pose.process(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
    status_text = "WAITING..."
    status_class = "status-idle"
    if results.pose_landmarks:
        lm = results.pose_landmarks.landmark
        nose_y = lm[0].y
        ankle_y = (lm[27].y + lm[28].y) / 2.0
        body_height = abs(ankle_y - nose_y)
        if min(visibility_scores) < 0.65 or body_height < 0.35:
            status_text = "STEP INTO FRAME"
            status_class = "status-warn"
            sequence_buffer.clear()
            squat_state = "up"
        else:
            mp_draw.draw_landmarks(frame, results.pose_landmarks)

```



```

features = []
for landmark in lm:
    features.extend([landmark.x, landmark.y, landmark.z])
current_knee_angle = (calculate_angle(lm[23], lm[25], lm[27]))
current_hip_angle = (calculate_angle(lm[11], lm[23], lm[25]))
st.session_state.current_knee_angle = current_knee_angle
target = st.session_state.target_angle
features.extend([current_knee_angle, current_hip_angle])
features.extend([0.0] * (240 - len(features)))
sequence_buffer.append(features)
sequence_buffer = sequence_buffer[-30:]
if current_knee_angle > 160 and current_hip_angle > 155:
    status_text = "IDLE"
    status_class = "status-idle"
    if squat_state == "down":
        reps += 1
        squat_state = "up"
        speak(f"{reps}")
else:
    if current_knee_angle <= (target + 15):
        squat_state = "down"
    if current_hip_angle < 75:
        status_text = "STRAIGHTEN YOUR BACK"
        status_class = "status-warn"
    elif current_knee_angle > (target + 15):
        status_text = "GO DOWN MORE"
        status_class = "status-warn"
    else:
        status_text = "PERFECT DEPTH"

```

```

        status_class = "status-good"
    current_time = time.time()
    if status_text not in ["WAITING...", "STEP INTO FRAME", "IDLE"]:
        if status_text != last_spoken_status or (current_time - last_spoken_time ):
            speak(status_text)
            last_spoken_status = status_text
            last_spoken_time = current_time
    if status_text in ["IDLE", "WAITING...", "STEP INTO FRAME"]:
        last_spoken_status = ""
    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    video_placeholder.image(frame_rgb, channels="RGB")

```

5.streamlit run app.py

```

import streamlit as st
import cv2
import numpy as np
import mediapipe as mp
from tensorflow.keras.models import load_model
st.set_page_config(page_title="PoseGuru AI", layout="wide",
initial_sidebar_state="collapsed")
st.markdown("""
<style>
    .stApp { background-color: #0f172a; color: #f8fafc; }
    .metric-card { background-color: #1e293b; padding: 20px; border-radius: 15px;
text-align: center; border: 1px solid #334155; margin-bottom: 15px; }
    .metric-value { font-size: 48px; font-weight: bold; color: #3b82f6; }
    .goal-value { font-size: 48px; font-weight: bold; color: #10b981; }
    .status-good { color: #10b981; font-size: 24px; font-weight: bold; }
    .status-warn { color: #ef4444; font-size: 24px; font-weight: bold; }
    .status-idle { color: #f59e0b; font-size: 24px; font-weight: bold; }

```

</style>

```
""", unsafe_allow_html=True)
```

```
def get_branch_features(x, landmark_indices):
```

```
    feature_indices = []
```

```
    for i in landmark_indices:
```

```
        feature_indices.extend([i*4, i*4+1, i*4+2, i*4+3])
```

```
    return tf.gather(x, feature_indices, axis=2)
```

```
def get_angle_features(x):
```

```
    return tf.gather(x, [132, 133], axis=2)
```

```
def calculate_angle(a, b, c):
```

```
    a = np.array([a.x, a.y])
```

```
    b = np.array([b.x, b.y])
```

```
    c = np.array([c.x, c.y])
```

```
    radians = np.arctan2(c[1]-b[1], c[0]-b[0]) - np.arctan2(a[1]-b[1], a[0]-b[0])
```

```
    angle = np.abs(radians * 180.0 / np.pi)
```

```
    return 360 - angle if angle > 180.0 else angle
```

```
@st.cache_resource
```

```
def load_ai():
```

```
    try:
```

```
        model = load_model("squat_specialist_v3.h5",
```

```
custom_objects={'get_branch_features': get_branch_features, 'get_angle_features':
```

```
get_angle_features, 'tf': tf})
```

```
    return model
```

```
    except Exception:
```

```
        return None
```

```
model = load_ai()
```

```
mp_pose = mp.solutions.pose
```

```
pose = mp_pose.Pose(min_detection_confidence=0.7, min_tracking_confidence=0.9)
```

```
mp_draw = mp.solutions.drawing_utils
```

```

st.title("POSEGURU | Clinical Rehab AI")
if 'target_angle' not in st.session_state:
    st.session_state.target_angle = 120.0
col1, col2 = st.columns([2, 1])
with col1:
    st.markdown("### Live Tracking")
    video_placeholder = st.empty()
    status_placeholder = st.empty()
with col2:
    st.markdown("### Clinical Metrics")
    reps_placeholder = st.empty()
    angle_placeholder = st.empty()
    goal_placeholder = st.empty()
    goal_placeholder.markdown(f"""
        <div class="metric-card">
            <p style="color: #94a3b8; letter-spacing: 2px;">ADAPTED GOAL</p>
            <p class="goal-value">{int(st.session_state.target_angle)}°</p>
        </div>
        """, unsafe_allow_html=True)
    start_btn = st.button("START CAMERA", use_container_width=True)
    stop_btn = st.button("STOP CAMERA", use_container_width=True)
    cal_btn = st.button("CALIBRATE SAFE DEPTH", use_container_width=True)
if cal_btn and 'current_knee_angle' in st.session_state:
    st.session_state.target_angle = st.session_state.current_knee_angle
    goal_placeholder.markdown(f"""
        <div class="metric-card">
            <p style="color: #94a3b8; letter-spacing: 2px;">ADAPTED GOAL</p>
            <p class="goal-value">{int(st.session_state.target_angle)}°</p>
        </div>
    """)

```

```

    """, unsafe_allow_html=True)
if start_btn:
    cap = cv2.VideoCapture(0)
    sequence_buffer = []
    squat_state = "up"
    reps = 0
    frame_count = 0
    st.session_state.current_knee_angle = 180.0
    while cap.isOpened() and not stop_btn:
        success, frame = cap.read()
        if not success:
            break
        frame = cv2.flip(frame, 1)
        frame_count += 1
        results = pose.process(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
        status_text = "WAITING..."
        status_class = "status-idle"
        if results.pose_landmarks:
            lm = results.pose_landmarks.landmark
            mp_draw.draw_landmarks(frame, results.pose_landmarks)
            if min([lm[23].visibility, lm[24].visibility, lm[25].visibility]) > 0.5:
                status_text = "BACK UP (SHOW LEGS)"
                status_class = "status-warn"
                sequence_buffer.clear()
            else:
                features = []
                for landmark in lm:
                    features.extend([landmark.x, landmark.y, landmark.z])
                current_knee_angle = (calculate_angle(lm[23], lm[25], lm[27]))

```

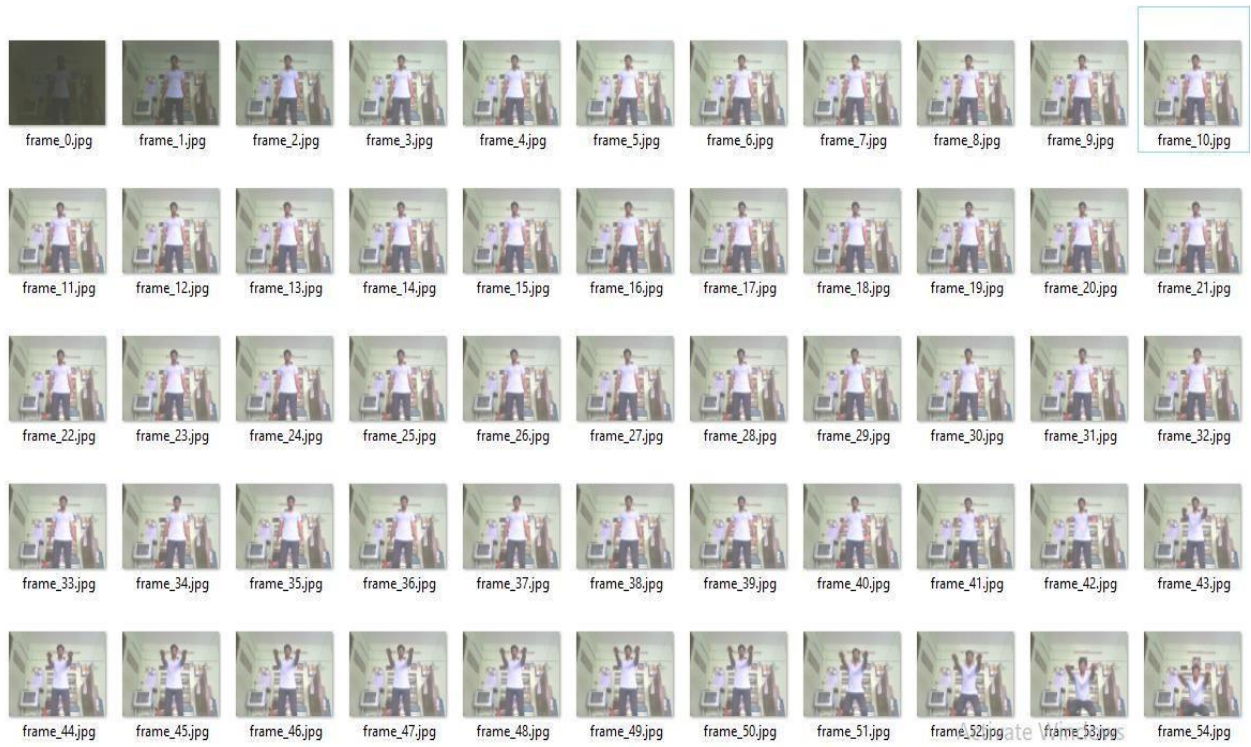
```

current_hip_angle = (calculate_angle(lm[11], lm[23], lm[25])
st.session_state.current_knee_angle = current_knee_angle
target = st.session_state.target_angle
features.extend([current_knee_angle, current_hip_angle])
features.extend([0.0] * (240 - len(features)))
sequence_buffer.append(features)
sequence_buffer = sequence_buffer[-30:]
if len(sequence_buffer) == 30 and frame_count % 4 == 0 and is not None:
    class_id = np.argmax(model.predict(np.expand_dims(sequence_buffer)
if current_knee_angle > 170 and current_hip_angle > 165:
    status_text = "IDLE"
    status_class = "status-idle"
    if squat_state == "down":
        reps += 1
        squat_state = "up"
else:
    squat_state = "down"
    if current_hip_angle < 100:
        status_text = "STRAIGHTEN YOUR BACK"
        status_class = "status-warn"
    elif current_knee_angle > (target + 5):
        status_text = "GO DOWN SLIGHTLY"
        status_class = "status-warn"
    else:
        status_text = "GOOD (ADAPTED REHAB)"
        status_class = "status-good"
frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
video_placeholder.image(frame_rgb, channels="RGB")
cap.release()

```

APPENDIX2: SCREEN SHOTS TRAINING PHASE

DATA ACQUISITION



POSE ESTIMATION



FEATURE EXTRACTION

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	
1	frame_id	timestamp	lm_0_x	lm_0_y	lm_0_z	lm_0_vis	lm_1_x	lm_1_y	lm_1_z	lm_1_vis	lm_2_x	lm_2_y	lm_2_z	lm_2_vis	lm_3_x	lm_3_y	lm_3_z	lm_3_vis	lm_4_x	lm_4_y	lm_4_z
2	1	1.161994	0.538438	0.269152	-0.48066	0.999993	0.548088	0.250546	-0.45952	0.999984	0.553651	0.250847	-0.45963	0.999981	0.558689	0.251289	-0.45977	0.999981	0.530241	0.251168	-0.
3	2	1.332836	0.538501	0.269209	-0.43835	0.999993	0.548232	0.250763	-0.41676	0.999984	0.553737	0.251039	-0.4169	0.999982	0.558871	0.251436	-0.41709	0.999982	0.530255	0.251232	-0.
4	3	1.469363	0.538708	0.269173	-0.38062	0.999993	0.548514	0.250732	-0.35731	0.999985	0.553901	0.250998	-0.35745	0.999983	0.559111	0.251381	-0.35762	0.999982	0.530523	0.251163	-0.
5	4	1.59437	0.538657	0.26889	-0.35446	0.999993	0.548494	0.250363	-0.33069	0.999985	0.553868	0.250641	-0.33084	0.999983	0.559051	0.251074	-0.331	0.999983	0.530501	0.250594	-0.
6	5	1.725878	0.538638	0.267721	-0.31576	0.999993	0.548494	0.249337	-0.2929	0.999985	0.553866	0.249698	-0.29306	0.999983	0.559052	0.250259	-0.29322	0.999983	0.530498	0.249234	-0.
7	6	1.825203	0.539043	0.266294	-0.36475	0.999993	0.54873	0.247974	-0.3409	0.999985	0.553928	0.248432	-0.34105	0.999984	0.559087	0.24908	-0.3412	0.999984	0.530938	0.247488	-0.
8	7	1.975996	0.538993	0.265465	-0.3404	0.999993	0.54866	0.24738	-0.31591	0.999985	0.553851	0.247901	-0.31605	0.999984	0.558989	0.248641	-0.31621	0.999984	0.53089	0.246791	-0.
9	8	2.140144	0.539095	0.263761	-0.33866	0.999993	0.548672	0.245994	-0.31361	0.999986	0.553847	0.246592	-0.31376	0.999984	0.558984	0.247387	-0.31391	0.999985	0.530944	0.245362	-0.
10	9	2.250902	0.538851	0.262792	-0.34107	0.999993	0.548451	0.245045	-0.32221	0.999986	0.55358	0.245708	-0.32234	0.999984	0.558629	0.24659	-0.32248	0.999984	0.530749	0.244308	-0.
11	10	2.37689	0.538824	0.262169	-0.34265	0.999993	0.548414	0.244254	-0.31865	0.999986	0.553463	0.244958	-0.3188	0.999984	0.558493	0.245839	-0.31893	0.999985	0.530733	0.243474	-0.
12	11	2.477147	0.538611	0.261489	-0.34749	0.999993	0.548192	0.243473	-0.32255	0.999986	0.553226	0.244199	-0.3227	0.999985	0.558252	0.245149	-0.32285	0.999985	0.530507	0.24263	-0.
13	12	2.677549	0.53798	0.26035	-0.3524	0.999993	0.547587	0.242352	-0.32703	0.999986	0.552608	0.243022	-0.3272	0.999984	0.557474	0.243983	-0.32735	0.999985	0.529898	0.241641	-0.
14	13	2.824888	0.537939	0.258845	-0.37151	0.999992	0.547521	0.240809	-0.34499	0.999985	0.552479	0.241434	-0.34514	0.999984	0.557374	0.242292	-0.34531	0.999985	0.52985	0.240172	-0.
15	14	3.016102	0.537865	0.258396	-0.28621	0.999992	0.547473	0.240025	-0.26138	0.999985	0.552418	0.240627	-0.26154	0.999983	0.557365	0.241448	-0.26173	0.999984	0.529773	0.239473	-0.
16	15	3.139459	0.538148	0.257801	-0.29305	0.999992	0.547651	0.23953	-0.26823	0.999984	0.552569	0.240175	-0.26838	0.999982	0.557745	0.240974	-0.26854	0.999983	0.530124	0.238587	-0.
17	16	3.283779	0.538564	0.257136	-0.30174	0.999991	0.547964	0.239054	-0.27659	0.999982	0.55291	0.23968	-0.27674	0.99998	0.558371	0.24045	-0.2769	0.999982	0.530583	0.23803	-0.
18	17	3.415333	0.539067	0.256814	-0.28663	0.99999	0.548389	0.238815	-0.26132	0.99998	0.553452	0.239415	-0.26147	0.999979	0.559165	0.240152	-0.26164	0.99998	0.530917	0.237662	-0.
19	18	3.548326	0.539983	0.25687	-0.31821	0.99999	0.548929	0.238831	-0.29374	0.999979	0.553994	0.239437	-0.29387	0.999978	0.559809	0.240189	-0.29408	0.99998	0.531622	0.237734	-0.
20	19	3.715682	0.539977	0.257487	-0.31236	0.999989	0.548965	0.239734	-0.28865	0.999978	0.554017	0.240301	-0.28878	0.999977	0.559844	0.240911	-0.28897	0.999978	0.531628	0.238493	-0.
21	20	3.82695	0.539963	0.257521	-0.32054	0.999989	0.548969	0.239756	-0.29729	0.999977	0.554018	0.240313	-0.29742	0.999976	0.559837	0.240937	-0.2976	0.999978	0.531626	0.238507	-0.
22	21	3.931275	0.539872	0.257376	-0.35493	0.999989	0.548955	0.239629	-0.33108	0.999977	0.553981	0.2402	-0.3312	0.999976	0.559807	0.240831	-0.33137	0.999978	0.531615	0.238436	-0.

TEMPORAL FILTER / LSTM TRAINING

```

22/22 [=====] - 0s 13ms/step - loss: 0.0262 - accuracy: 0.9882 - val_loss: 0.0480 - val_accuracy: 0.9765
Epoch 32/40
22/22 [=====] - 0s 13ms/step - loss: 0.0293 - accuracy: 0.9853 - val_loss: 0.0504 - val_accuracy: 0.9647
Epoch 33/40
22/22 [=====] - 0s 13ms/step - loss: 0.0250 - accuracy: 0.9912 - val_loss: 0.0385 - val_accuracy: 0.9882
Epoch 34/40
22/22 [=====] - 0s 13ms/step - loss: 0.0274 - accuracy: 0.9882 - val_loss: 0.0707 - val_accuracy: 0.9647
Epoch 35/40
22/22 [=====] - 0s 13ms/step - loss: 0.0327 - accuracy: 0.9912 - val_loss: 0.0370 - val_accuracy: 0.9765
Epoch 36/40
22/22 [=====] - 0s 13ms/step - loss: 0.0306 - accuracy: 0.9794 - val_loss: 0.0393 - val_accuracy: 0.9765
Epoch 37/40
22/22 [=====] - 0s 13ms/step - loss: 0.0234 - accuracy: 0.9853 - val_loss: 0.0383 - val_accuracy: 1.0000
Epoch 38/40
22/22 [=====] - 0s 13ms/step - loss: 0.0228 - accuracy: 0.9912 - val_loss: 0.0644 - val_accuracy: 0.9765
Epoch 39/40
22/22 [=====] - 0s 13ms/step - loss: 0.0424 - accuracy: 0.9882 - val_loss: 0.1368 - val_accuracy: 0.9294
Epoch 40/40
22/22 [=====] - 0s 13ms/step - loss: 0.0333 - accuracy: 0.9794 - val_loss: 0.0793 - val_accuracy: 0.9765

```

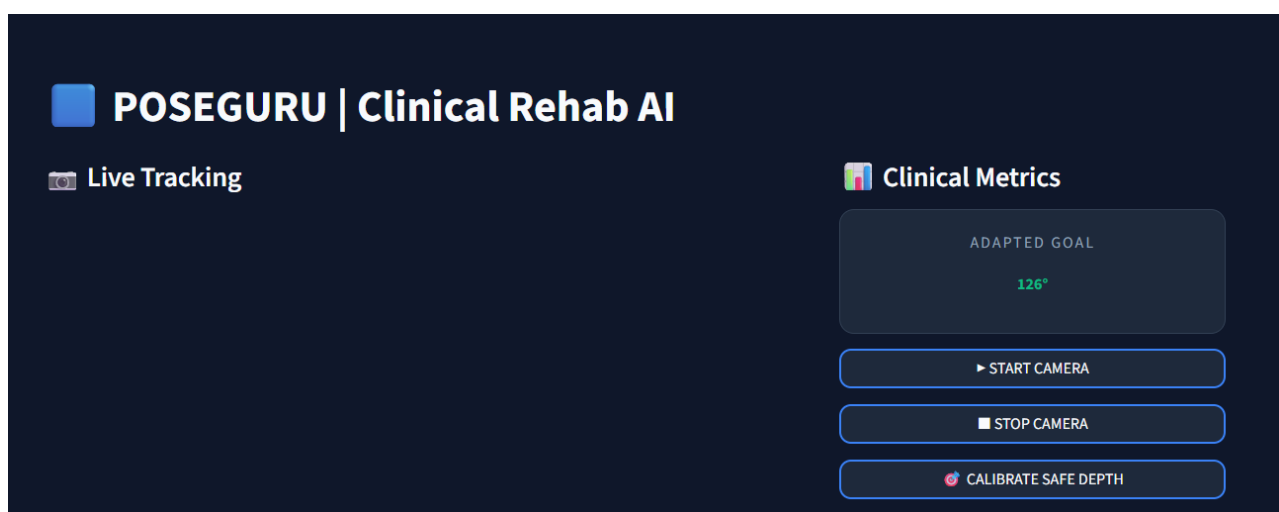

CALIBRATE

```
PS D:\PoseGuru_Project1> py -3.11 calibrate_user.py
2026-03-01 22:05:04.136252: I tensorflow/core/util/port.cc:113] oneDNN
cal results due to floating-point round-off errors from different comp
TF_ENABLE_ONEDNN_OPTS=0`.
WARNING:tensorflow:From C:\Users\ELCOT\AppData\Local\Programs\Python\P
.losses.sparse_softmax_cross_entropy is deprecated. Please use tf.comp

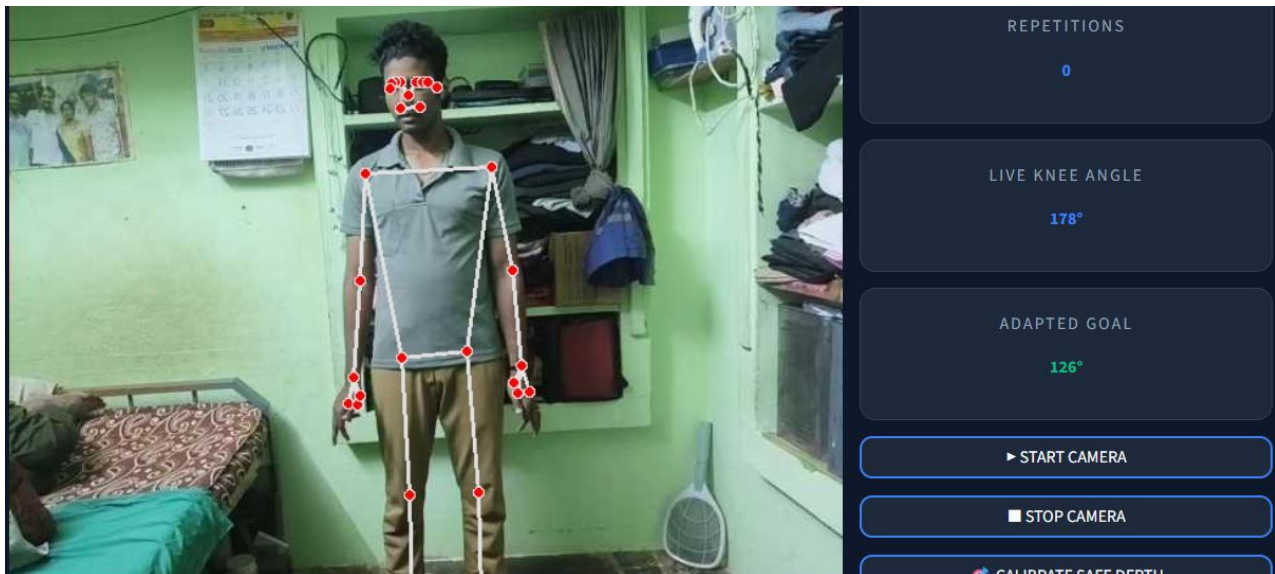
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
WARNING: All log messages before absl::InitializeLog() is called are w
W0000 00:00:1772382915.519922 20844 inference_feedback_manager.cc:11
erence. Disabling support for feedback tensors.
W0000 00:00:1772382915.556654 20844 inference_feedback_manager.cc:11
erence. Disabling support for feedback tensors.
C:\Users\ELCOT\AppData\Local\Programs\Python\Python311\Lib\site-packag
abase.GetPrototype() is deprecated. Please use message_factory.GetMess
d soon.
  warnings.warn('SymbolDatabase.GetPrototype() is deprecated. Please '
✓ Target saved: 149°
PS D:\PoseGuru_Project1>
```

EXECUTION PHASE

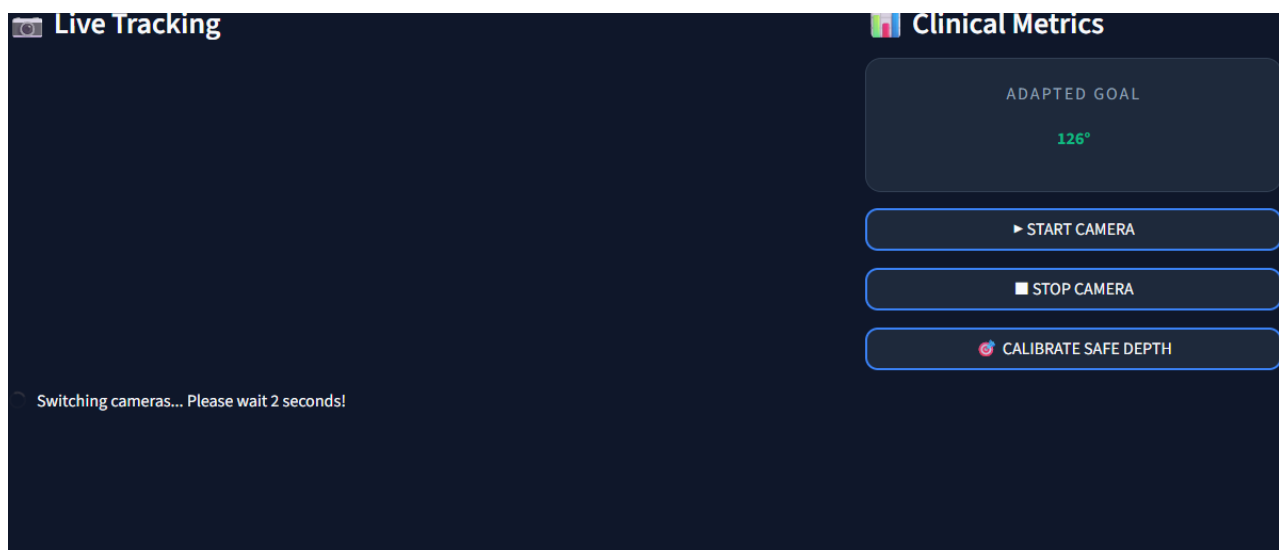
APPLICATION DASHBOARD



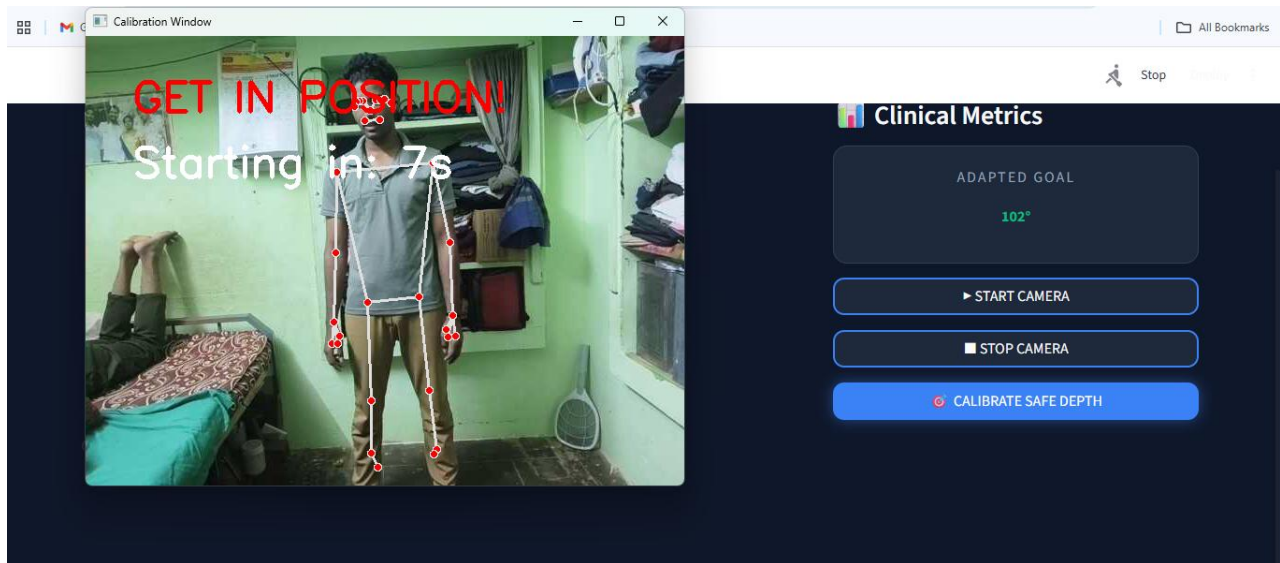
START CAMERA



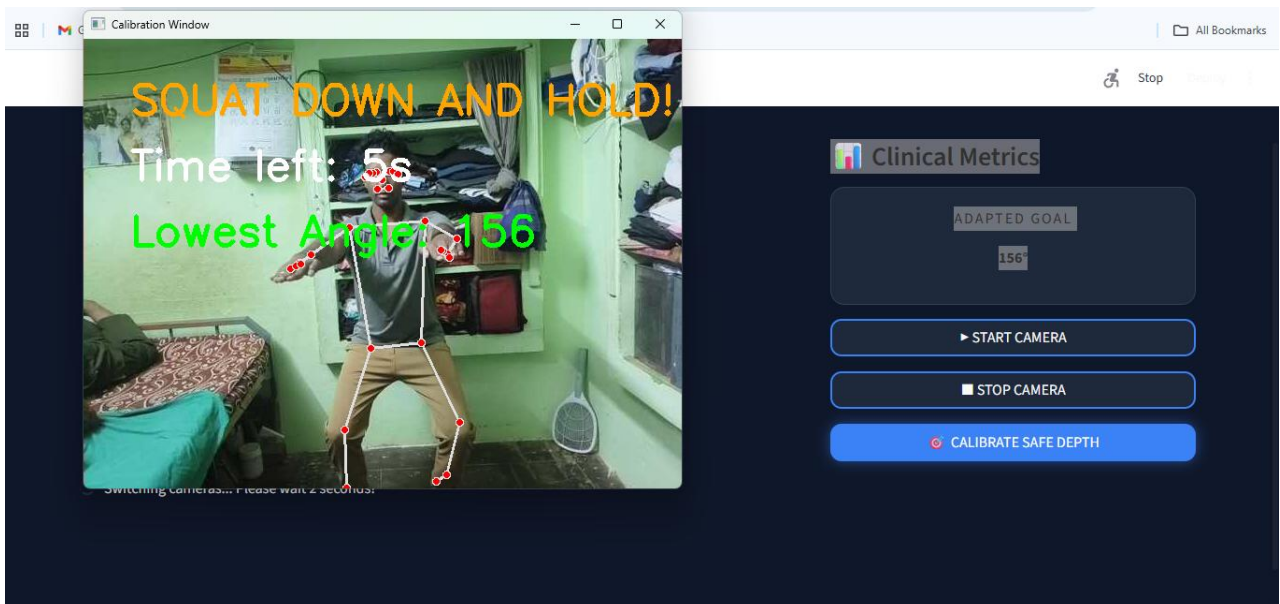
CALIBRATE SAFE DEPTH



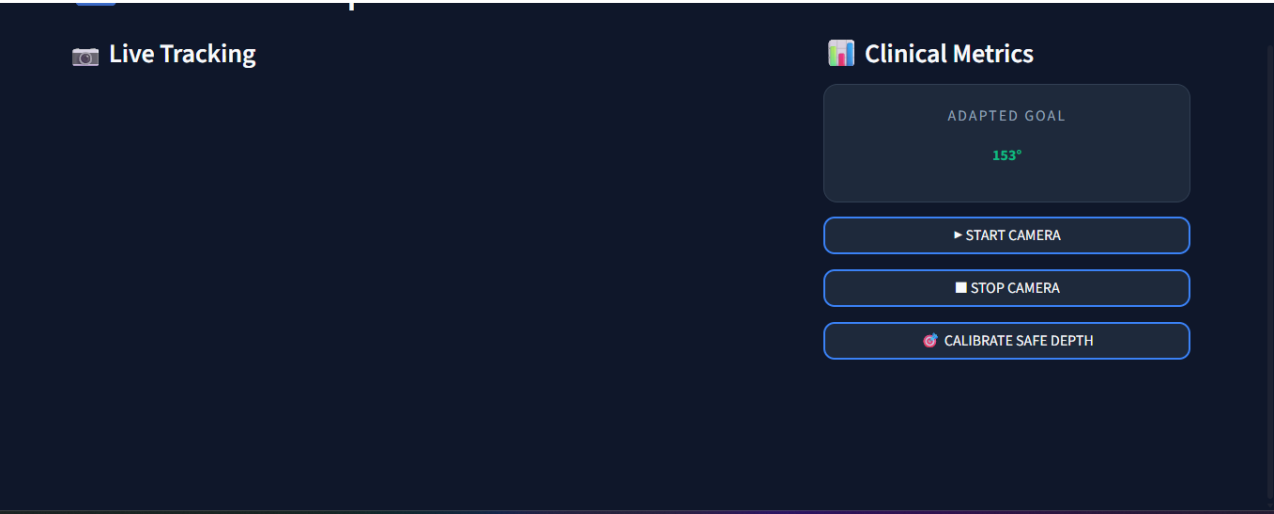
PREPARE FOR SQUAT



PERFORM EXERCISE

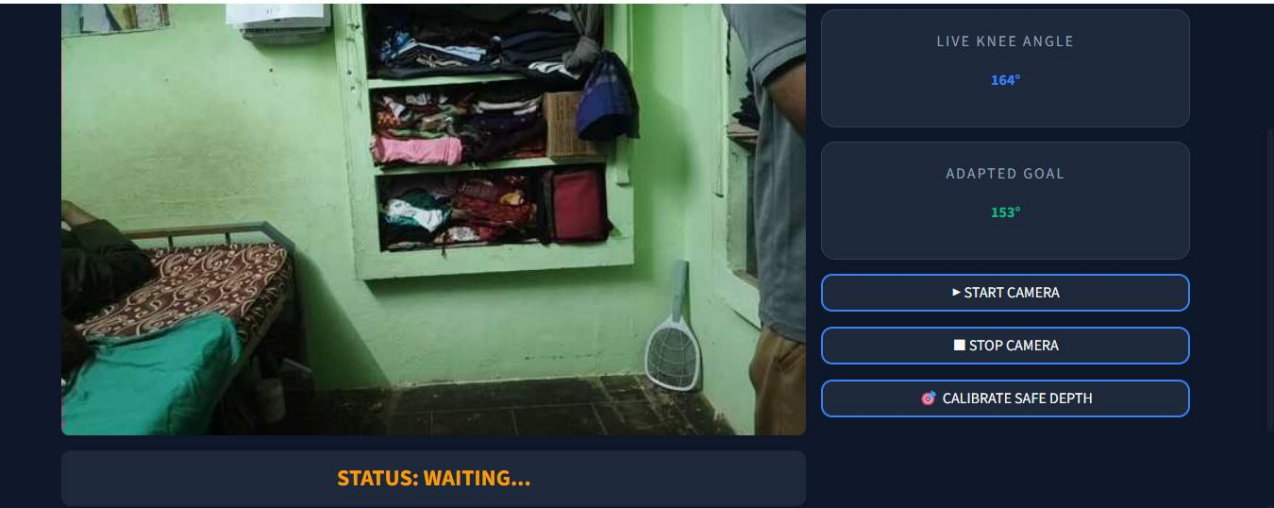


CALIBRATION ANGLE UPDATED



STATUS UPDATION

WAITING



IDLE



LIVE KNEE ANGLE
177°

ADAPTED GOAL
153°

► START CAMERA

■ STOP CAMERA

🔧 CALIBRATE SAFE DEPTH

STATUS: IDLE

STEP INTO FRAME



LIVE KNEE ANGLE
162°

ADAPTED GOAL
153°

► START CAMERA

■ STOP CAMERA

🔧 CALIBRATE SAFE DEPTH

STATUS: STEP INTO FRAME

INSTRUCTIONS



LIVE KNEE ANGLE
158°

ADAPTED GOAL
114°

► START CAMERA

■ STOP CAMERA

🔧 CALIBRATE SAFE DEPTH

STATUS: GO DOWN MORE

PERFECT POSE



158°

ADAPTED GOAL
153°

► START CAMERA

■ STOP CAMERA

🔧 CALIBRATE SAFE DEPTH

STATUS: PERFECT DEPTH

BIBLIOGRAPHY

- [1] A. Dittakavi et al. "PoseGuru: Landmarks for Explainable Pose Correction using Exemplar-Guided Algorithmic Recourse (CVPRW-2025)".
- [2] Lei Yang, Yingxiang Li, Degui Zeng, Dong Wang. "Human Exercise Posture Analysis based on Pose Estimation (IEEE-2021)".
- [3] Ali Bidaran, Saeed Sharifian. "Designing an AI-assisted toolbox for fitness activity recognition based on deep CNN (IEEE-2021)".
- [4] Wan-Jung Chang, Jian-Ping Su, Chia-Hao Chen, et al. "iFitness: A Deep Learning-Based Physical Fitness Motion Detection System for Elderly People (IEEE-2021)".
- [5] Prashant Rao, C. S. Asha, P. Raghavendra Rao. "Real-Time Posture Correction of Squat Exercise: A Deep Learning Approach for Performance Analysis and Error Correction (IEEE-2024)".
- [6] Aisha Naseer, Ali Raza, Hadeeqa Afzal, et al. "Human Pose Estimation in Physiotherapy Fitness Exercise Correction Using Novel Transfer Learning Approach (PeerJ-2024)".
- [7] Ulises Gallardo, Fernando Caro, Eluney Hernández, et al. "GYMetricPose: A Light-Weight Angle-Based Graph Adaptation for Action Quality Assessment (IEEE-2024)".
- [8] Mukundan Chariar, Shreyas Rao, Aryan Irani, et al. "AI Trainer: Autoencoder Based Approach for Squat Analysis and Correction (IEEE-2023)".